ATLAS

Version 3.0

User Manual

Cohort definition and analytics on OMOP CDM data

Peter Hoffmann

First Edition — May 6, 2026

Preface

This is the user manual for ATLAS v3.0, the next version of the OHDSI ATLAS web application. It is a working manual: it describes what each area of the application does, how to use it, and the ATLAS-v3.0-specific behaviour that a researcher meets at the keyboard — the configuration panel, the version-history tabs, the cohort-builder controls, the plugin surface, the auto-save semantics — in the order a user encounters them.

This manual is *complementary* to *The Book of OHDSI* [20], not a replacement for it. The Book is the canonical reference for the methodology behind every ATLAS feature: the OMOP Common Data Model, the standardised vocabularies, phenotype development, characterization, comparative effectiveness, clinical and method validity. Where this manual touches one of those topics, it does so only enough to name the ATLAS controls a user is looking at, and points at the relevant Book chapter for the rest. Read them alongside each other.

Audience

The manual serves several readers:

- **Researchers and analysts** who design cohort definitions, run characterizations, pathways, or incidence rate analyses, and read the resulting reports.
- **Clinicians and domain experts** reviewing concept sets and cohort designs for clinical validity — they need enough of the interface vocabulary to comment usefully without becoming ATLAS operators.
- **Administrators** configuring data sources, managing roles, and supporting users in their organisation.
- **New OHDSI participants** for whom ATLAS v3.0 is their first contact with the OMOP Common Data Model and the OHDSI toolchain.

What this manual is, and is not

This is a *task-oriented* manual. Each chapter takes one feature, explains what it lets you do, and walks through the workflow. Where a screen requires deeper conceptual background — the difference between a standard concept and a source code, why a cohort is more than a list of people — the manual stops to explain rather than glossing over. Cross-references rather than re-explanations are used when a concept reappears in a later chapter.

This is not an installation handbook for first-time deployment; the appendix gives a working setup recipe but the canonical instructions live in the project's repository. It is

also not a WebAPI reference. The manual documents the user interface; if you need to call WebAPI directly — to script cohort generation, batch concept-set imports, or build a plugin against the same endpoints the UI uses — the appendix points to WebAPI's live swagger UI, which is the authoritative endpoint reference. Per-section endpoint footnotes are deliberately not included.

Structure

The chapters follow the order in which most users meet the features:

- **Foundations** (Chapters 1–2) — what ATLAS v3.0 is for, and how to sign in and navigate.
- **Building blocks** (Chapters 3–5) — data sources, vocabulary search, and concept sets, the components every later workflow draws on.
- **Cohort definitions** (Chapter 6) — the longest single workflow in the application: entry events, inclusion criteria, exit, generation, reports, and the diagnostics-and-evaluation cycle the OHDSI community wraps around it.
- **Analyses** (Chapters 7–11) — characterizations, cohort pathways, incidence rates, patient profiles (including phenotype validation with PheValuator), and reusable feature analyses.
- **Operations** (Chapters 12–14) — versioning, administration, and the plugin system.
- **Appendix A** — a setup recipe for WebAPI v3.0 and the ATLAS v3.0 frontend, plus pointers to the live WebAPI reference and the wider OHDSI toolchain.

Conventions

Visible UI labels are bold (**Save**). Routes, configuration keys, and code identifiers appear in monospace (`/cohorts`). Menu paths use a sans-serif arrow form (Analysis → Characterizations). Screenshots are shown framed; until real captures replace them, slots are marked with a placeholder caption that names the figure to come. Subdued left-rule blocks distinguish notes, tips, cautions, and concept side-bars from the running prose.

A short cheat sheet of the OHDSI ecosystem (Achilles, HADES, PheValuator, CohortDiagnostics, Athena, Broadsea, Phenotype Library, Themis) is in the introduction's "OHDSI toolchain at a glance" notebox. A glossary of OMOP / OHDSI terminology (cohort, concept, era, index date, standard concept, etc.) is at the back of the manual.

Acknowledgments

ATLAS v3.0 is the work of the OHDSI community continuing the long tradition of the original ATLAS. This manual draws on the OHDSI ATLAS wiki [21], *The Book of OHDSI* [20], and on the ATLAS v3.0 codebase itself. Any error in description belongs to the manual; the application owes its shape to its many contributors.

Contents

PART I

Foundations

CHAPTER 1

Introduction

ATLAS v3.0 is a web application for designing, generating, and reviewing patient cohorts on data that conforms to the OMOP Common Data Model (CDM) [4, 16, 29]. It is a modern reimplementation of the long-standing OHDSI ATLAS tool. Functionally it covers the same ground as ATLAS 2.x — vocabulary search, concept sets, cohort definitions, characterizations, cohort pathways, incidence rates and patient profiles — built on top of the same WebAPI backend. The user interface has been rebuilt with Vue 3 [43] and Vuetify [8], and several workflows have been streamlined.

This manual is task-oriented. Each chapter takes one feature, explains what it lets you do, and walks through it in the order a user would meet it. If you are completely new to OHDSI, read this introduction and Chapter 2 first. If you already know ATLAS, you can jump straight to the chapter that matches the feature you need.

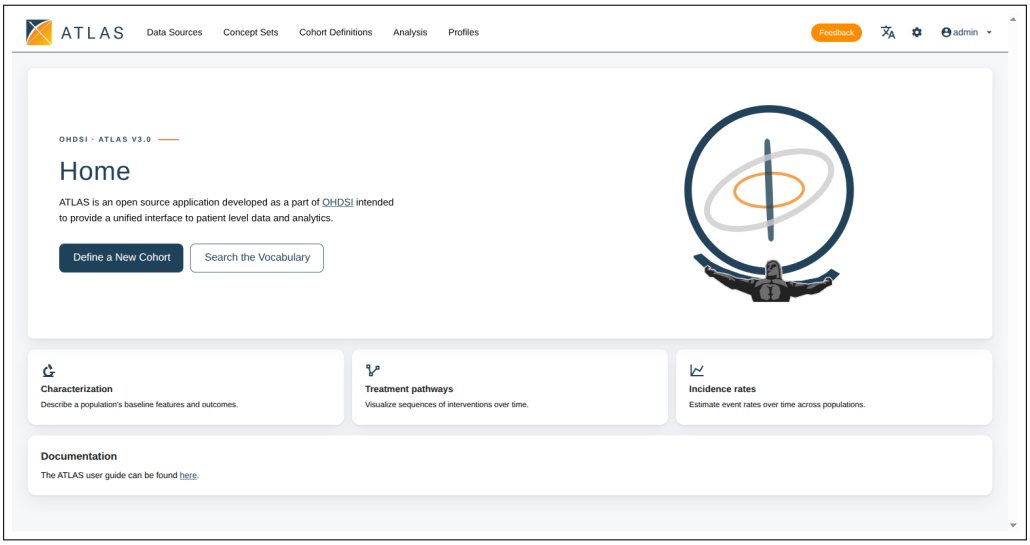


Figure 1.1. The ATLAS v3.0 home page. The top navigation lists the major application areas; the home card provides the two most-used entry points (**Define a New Cohort**, **Search the Vocabulary**) plus a row of analysis-area shortcuts.

1.1 What ATLAS v3.0 is for

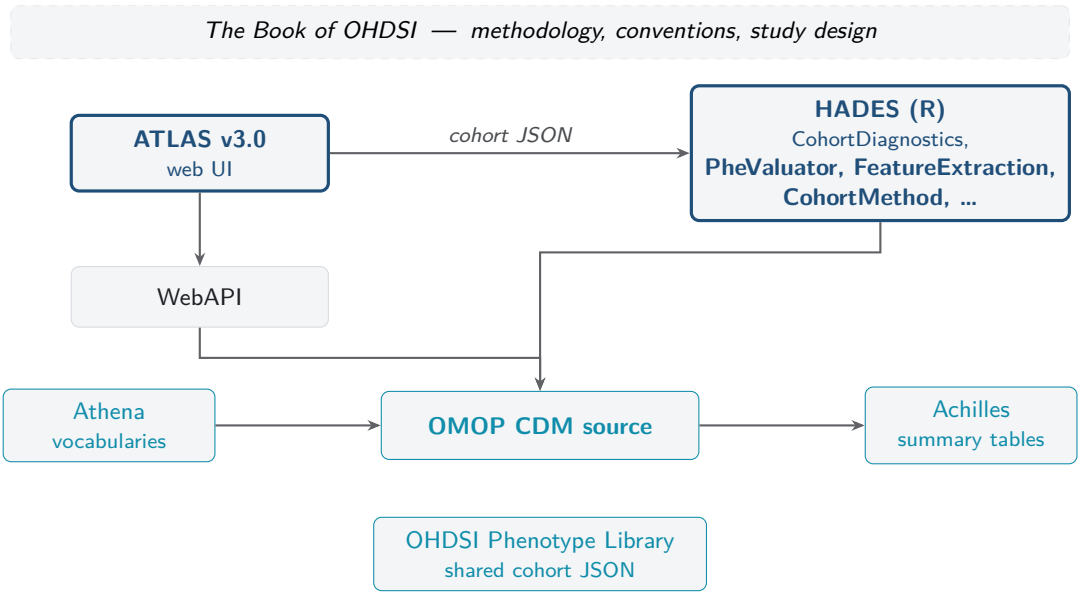
Observational health data — claims, electronic health records, registries — answer different questions than clinical trials. They are large, messy, and the records were collected for billing or care, not research. The OHDSI community addresses this with two pieces: a standardised data model (the OMOP CDM) and a library of methods that operate on that model. ATLAS v3.0 sits on top of both. It is the place where a researcher:

- explores which data sources are available and what they contain;
- searches the standardised vocabularies for the concepts that describe a clinical idea (a disease, a drug class, a procedure);
- collects those concepts into reusable *concept sets*;
- composes *cohort definitions* — formal, executable descriptions of a population such as "new users of metformin without a prior diagnosis of diabetes";
- generates the cohort against one or more data sources and reviews descriptive reports about it;
- runs additional analyses — characterizations, pathways, incidence rates — using cohorts as inputs.

Everything ATLAS v3.0 produces is a portable artefact. A cohort definition is stored as JSON and SQL and can be shared, version-controlled, and run again on a different network on a different CDM database to reproduce a finding. The OHDSI design ethos is to *execute analytics in place*: the patient-level data never leaves the hosting institution, and only aggregate results are pooled across the network [20].

The cohort builder in ATLAS v3.0 is *rule-based*: a person is in the cohort if and only if they satisfy a set of explicit, hand-crafted criteria. Probabilistic phenotyping — where a model assigns each person a probability of meeting the phenotype — is a separate practice supported by the HADES PheValuator package (Chapter 11) [37].

The OMOP CDM is a person-centric relational schema. Every record about every person — a condition, a drug exposure, a measurement result — is stored in a small, predictable set of tables, and every clinical idea is represented by a *concept* drawn from the OHDSI standardised vocabularies. Because the schema and the vocabularies are the same everywhere, the same cohort definition you build in ATLAS v3.0 produces comparable results on a Korean claims database, a Dutch GP record system, and a US academic medical centre [41]. See *The Book of OHDSI*, chapter "OHDSI Analytics Tools" [20] for the longer treatment.



How ATLAS v3.0 sits in the OHDSI ecosystem. Solid arrows are runtime data flows. Two further relationships are not drawn to keep the figure readable: ATLAS reads Achilles' summary tables when it renders descriptive reports, and ATLAS imports cohort JSON from the Phenotype Library.

Note

The OHDSI toolchain at a glance. ATLAS is one piece of a larger OHDSI ecosystem. The names below are all referenced elsewhere in this manual; this is the cheat sheet:

ATLAS this web application; designs and runs cohort definitions and analyses.

WebAPI the Java/Spring backend ATLAS talks to. It compiles a cohort definition into SQL and executes it against a CDM source.

OMOP CDM the standardised relational schema all CDM sources conform to.

Athena the OHDSI vocabulary repository. The standardised vocabularies are released here.

Achilles an R package that profiles a CDM source and writes summary tables. ATLAS's descriptive reports read from those tables — if Achilles has not been run, the Data Sources page is blank.

HADES the OHDSI suite of R analytical packages (CohortMethod, FeatureExtraction, PatientLevelPrediction, and the two below).

CohortDiagnostics HADES package that runs deeper diagnostics on a cohort definition exported from ATLAS.

PheValuator HADES package that estimates phenotype sensitivity, specificity, and PPV against a probabilistic gold standard.

OHDSI Phenotype Library community-curated repository of vetted cohort definitions, each release DOI'd.

Themis the OHDSI conventions group; defines best practices for CDM and cohort design.

Broadsea a Docker bundle of the OHDSI stack (currently ATLAS 2.x; v3.0 support pending).

1.2 What ATLAS v3.0 is not

ATLAS v3.0 is a *design and review* tool. It does not store the patient data itself, and it does not run advanced analytics in the browser. When you press **Generate**, ATLAS v3.0 sends the cohort definition to WebAPI, which compiles it to SQL and runs it against the chosen CDM database.

The Population-Level Estimation and Patient-Level Prediction [33] modules from older ATLAS versions are *not* part of ATLAS v3.0 and there is no concrete plan to re-implement them in this codebase. For comparative effectiveness and prediction work, use the OHDSI HADES Libraries [23] directly in R against the same CDM data, or fall back to the classic ATLAS instance your organisation maintains.

1.3 How this manual is organised

- **Chapter 2, Getting Started** — what you need before you log in, how to log in, and a tour of the main navigation.
- **Chapter 4, Data Sources** — picking a CDM database and reading its descriptive reports.
- **Chapter 5, Vocabulary Search** — searching the OMOP standardised vocabularies.
- **Chapter 6, Concept Sets** — building and sharing reusable collections of concepts.
- **Chapter 7, Cohort Definitions** — the longest chapter; covers entry events, inclusion rules, exit, censoring, generation, and reports.
- **Chapters 8–10** — the analytical modules: characterizations, pathways, incidence rates.
- **Chapter 11, Profiles** — exploring an individual patient timeline.
- **Chapter 12, Feature Analyses** — reusable covariate definitions consumed by characterizations.
- **Chapter 13, Versions** — saving, previewing and restoring previous versions of cohorts and concept sets.
- **Chapter 14, Administration** — roles and permissions for users with administrative access.
- **Chapter 15, Plugins** — what plugins are and how they appear inside ATLAS v3.0.

1.4 Conventions used

- UI labels are bold: click **Save**, open the **Concept Sets** tab.
- Menu paths use chevrons: Analysis › Characterizations.
- Routes and code are monospaced: `/cohorts`, the JSON key `conceptSet`.
- Side notes appear in coloured boxes:

Note

A short fact worth knowing but not essential to the procedure being described.

A faster way to do the same thing.

Something that can ruin your work or produce misleading results if you are not careful.

Throughout the manual, where a section maps onto a single WebAPI endpoint or store action, that information appears as a small footnote at the end of the section so power users know where the UI's data is coming from.

CHAPTER 2

Getting started

This chapter assumes someone has already installed ATLAS v3.0 and pointed it at a WebAPI instance — installation is covered in the project's repository README. Here we look at the experience from a new user's perspective: what you need on day one, how to log in, and what each main area of the application is for.

2.1 Before you log in

You need three things:

1. A URL for the ATLAS v3.0 instance your organisation hosts. In Docker development this is usually `https://localhost`.
2. Credentials, or an identity provider (LDAP, OAuth, SAML, OpenID Connect) that the WebAPI is configured to accept.
3. At least one CDM database that WebAPI is connected to — without a data source, every analytical feature is read-only and most lists will be empty.

Note

Public OHDSI demonstration instances such as `https://atlas-demo.ohdsi.org` expose synthetic data. They are useful for trying ATLAS v3.0 risk-free but the population sizes there are not representative of the data you would use for a real study.

2.2 Signing in

Open the ATLAS v3.0 URL in a modern browser (Chromium-based browsers and Firefox are the test targets). ATLAS v3.0 lets you browse some pages unauthenticated, but every action that reads or writes WebAPI data requires a session, so the login dialog appears the first time you click into a feature like **Cohort Definitions**.

What appears in the login dialog depends on what the WebAPI is configured to accept:

Username and password A simple form. Used by LDAP-backed deployments and by the local user database.

OAuth providers One button per configured provider, e.g. Google. Clicking opens the provider's consent screen and returns to ATLAS v3.0.

SAML / OpenID Connect Same flow, used in enterprise single sign-on.

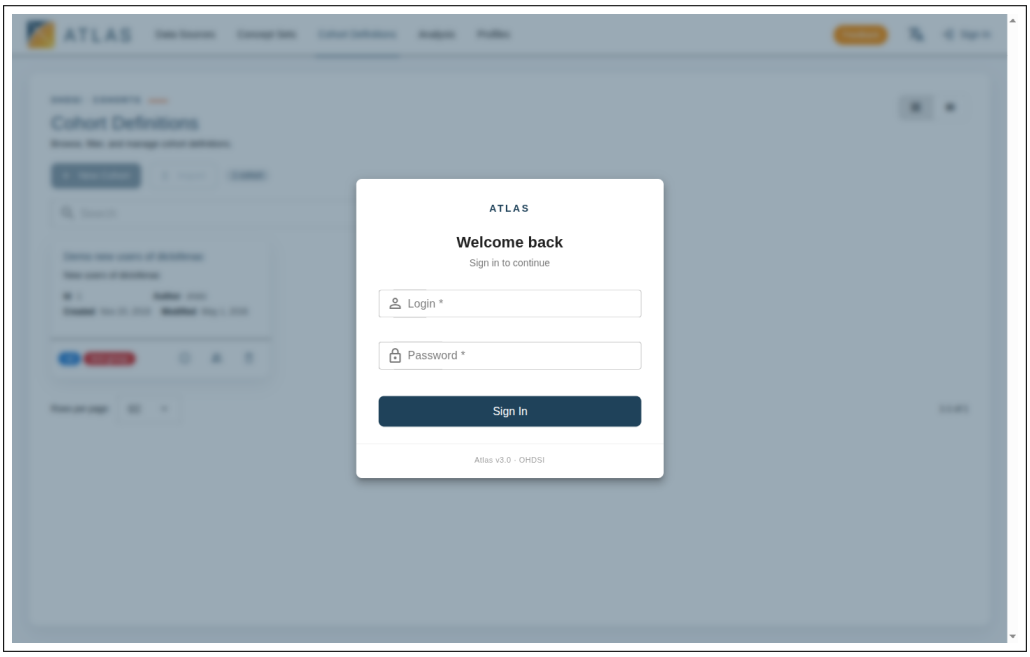


Figure 2.1. Login dialog. Provider buttons appear when the deployment is configured for OAuth, SAML, or OpenID; here only the database-credentials form is enabled.

After a successful sign-in your username appears in the top right. Click it to sign out.

2.2.1 Sessions and how they expire

ATLAS v3.0 stores a JWT bearer token in the browser. The token has a fixed lifetime; a few minutes before it expires ATLAS v3.0 shows a dialog inviting you to extend the session. If you walk away from your machine the token quietly expires and the next request triggers the login dialog again. No unsaved work is lost — ATLAS v3.0 keeps an in-progress cohort definition in session storage and restores it after you sign back in.

If you keep getting logged out unexpectedly, check the system clock on the WebAPI server. A clock skew larger than the token's clock-tolerance makes every freshly issued token look already expired.

2.3 The main navigation

The top of the screen has a navigation bar with the major application areas, plus a configuration menu for users with administrative access. The left sidebar (where present) gives a chapter-by-chapter breakdown of the area you are currently in.

Figure 2.2. The top navigation bar. From left: the ATLAS logo, the five built-in entries, then the right-hand cluster of Feedback button, language selector, configuration cog (visible to admins only), and the user menu.

The top navigation bar contains five built-in entries plus a small control cluster on the right. From left to right:

Data Sources The list of connected CDM databases, with a descriptive report per source. Covered in Chapter 4.

Concept Sets Browse, search and edit reusable collections of concepts. Vocabulary search lives inside the same area, on a sibling tab. Covered in Chapter 5 and Chapter 6.

Cohort Definitions List, search, filter, edit and generate cohorts. Covered in Chapter 7.

Analysis A tabbed area combining Analysis › Characterizations, Analysis › Cohort Pathways, Analysis › Incidence Rates and Analysis › Feature Analyses. Each is its own chapter (8–12).

Profiles Per-patient timeline view. Reached either directly from the top nav, or via a "view profile" action from another page. Covered in Chapter 11.

The right-hand control cluster carries a **Feedback** button (opens an external feedback form configured by the deployment), a language selector, the configuration cog, and the user menu. The cog is only shown when the signed-in user holds at least one administrative permission — if you cannot see it, ask your administrator. Plugins your organisation has installed extend this navigation with additional entries (Chapter 15).

Note

Permissions affect what you see. Many actions in ATLAS v3.0 are gated by per-entity permissions: you may see a cohort or concept set in a list but find that **Save** or **Delete** is disabled, and the configuration cog is hidden entirely if you do not have any admin permission. Chapter 14 describes the role and permission model.

A few features live outside the top nav and are reached by other means:

Home Click the ATLAS logo on the far left to return to the landing page at any time.

Jobs Long-running task status lives inside the configuration panel (cog icon) under the Jobs tab.

2.4 Switching language

ATLAS v3.0 is internationalised through the WebAPI translation bundle. Open the user menu and pick a language; the interface re-renders without reloading the page. The text of cohort definitions and concept sets is stored verbatim — only the chrome around them is translated.

2.5 A first walkthrough

To get a feel for the application, the OHDSI community recommends this short loop on a demo dataset:

1. Open **Data Sources** and pick any source. Wait for the descriptive report to load and skim the demographics chart.
2. Open **Concept Sets**, switch to the **Concept Search** tab and search the vocabulary for a familiar condition (e.g. "atrial fibrillation"), add a couple of concepts to a new set, tick **Descendants** on each row in the **Included Concepts** tab and save.
3. Open **Cohort Definitions**, click **New Cohort**, set your concept set as the entry event, leave inclusion criteria empty, and press **Generate** against the same data source.
4. When generation finishes, the **Generation** side panel shows the inclusion-rule statistics — skim them, then drill in to the demographics breakdown.

That round-trip — vocabulary → concept set → cohort → report — is the core motion of ATLAS v3.0. The rest of the manual is variations on it.

CHAPTER 3

A first phenotype: end-to-end

This chapter walks through one complete piece of work in ATLAS v3.0, from logging in to reading the cohort report. The reference chapters that follow describe each feature in depth; this one shows how they connect. If you are new to ATLAS, do this walkthrough on the public demo instance (`https://atlas-demo.ohdsi.org`) before tackling your own study.

Note

The demo instance ships a synthetic source called `SYNPUF1K` that is small enough to generate against in seconds. The exact counts you see may differ slightly from the figures below if the demo instance refreshes its sample, but the *shape* of the result will be the same.

3.1 The phenotype

Goal: *new users of metformin who do not have a prior diagnosis of diabetes recorded in the year before they start the drug*. This is a deliberately simple definition that touches every part of ATLAS: vocabulary search, two concept sets, an entry event with a washout, one inclusion rule using a zero-cardinality "absence" pattern, and a generation against the demo source.

3.2 Step 1 — Confirm the source is available

Open **Data Sources** from the top navigation. Pick `SYNPUF1K` from the source dropdown; the **Dashboard** report should render demographics and an observation-period chart. If the dashboard is empty, see Chapter 16.

3.3 Step 2 — Build the metformin concept set

1. Open **Concept Sets** from the top navigation; the **Concept Sets** list-tab is the default. Click **New Concept Set**; the editor opens as a side drawer. Name it `Metformin --- ingredient` in the inline title at the top.
2. In the editor drawer, switch to the **Search** tab. Type `metformin` into the search hero and press `Enter`.

3. In the results table, sort or scan to find the row whose name is exactly *metformin*, vocabulary `RxNorm`, type *Standard*, class *Ingredient* (concept ID 1503297). Click **Add** on that row.
4. Switch back to the **Included Concepts** tab. Tick **Descendants** on the row. Leave **Mapped** off and **Exclude** off.
5. Skim the row list — with descendants the set should cover branded and generic metformin products.
6. Click **Create** (or **Save**) on the drawer header to persist the set.

3.4 Step 3 — Build the diabetes concept set

The same flow with a different parent concept:

1. Click **New Concept Set**, name it `Diabetes mellitus`.
2. On the **Search** tab, search `diabetes mellitus`; find the SNOMED standard concept whose name is exactly *Diabetes mellitus* (concept ID 201820). Click **Add**.
3. Switch to the **Included Concepts** tab and tick **Descendants** on the row so all subtypes are covered.
4. Click **Create**.

3.5 Step 4 — Create the cohort definition

1. Open **Cohort Definitions** from the top navigation. Click **New Cohort**, name it `New users of metformin without prior diabetes`, and save.
2. In step 1 of the editor (**Cohort Entry Events**), click **Add criteria to group...** and pick **Add Drug Exposure**.
3. In the entry-event card, click the concept-set picker and choose `Metformin --- ingredient`.
4. Click the observation-period chip ("0 days before and 0 days after the event index date") to open its dialog; set the prior days to 365, leave post days at 0, and close the dialog.
5. Set the **Cohort entry on** toggle (top-right of step 1) to **earliest event** so that only each person's first qualifying drug exposure counts.

3.6 Step 5 — Add the inclusion rule

If the demo source's patient cache is built, the patient-count bar above the editor shows a running cohort estimate as you add the inclusion rule below — the number should drop noticeably when the "no prior diabetes" rule is applied. If it does not drop visibly, either the source has very few diabetes records or the concept set has not resolved correctly; treat the live count as a sanity check.

1. In step 2 (**Inclusion Criteria**), click **Add rule**. Name the new rule `No prior diabetes diagnosis`.
2. Inside the rule, click **Add criteria to group...** and pick **Add Condition Occurrence**.
3. For the concept set, pick `Diabetes mellitus`.
4. In the temporal window, choose "365 days before index" to "0 days before index".
5. In the cardinality dropdown, pick *exactly 0 occurrences*. This is the no-negation pattern from Chapter 7: there is no NOT operator; absence is expressed as a zero-cardinality rule.

3.7 Step 6 — Set cohort exit

In the **Cohort exit** section, choose **End of continuous drug exposure** so the person is in the cohort for as long as they keep getting metformin prescriptions (allowing a 30-day persistence window between consecutive fills). Pick `Metformin --- ingredient` as the concept set.

3.8 Step 7 — Generate

Click **Save** to persist the cohort, then click **Generate** on the cohort toolbar. The **Generation** side panel opens; in the data-source tile grid on the left, click the `SYNPUF1K` tile to start the run. The tile shows progress; on the demo source the run completes in seconds.

Note

If the run finishes but everything is blank, or the tile sits at **RUNNING** forever, the troubleshooting chapter (Chapter 16) covers the usual culprits: source not Achilles-processed, a patient cache that needs to be built first, no rows because the concept set resolves to nothing on this source, or an attribute filter that excludes everyone.

3.9 Step 8 — Read the inclusion-rule report

When the `SYNPUF1K` tile shows **COMPLETED**, click it. On the right of the Generation panel, the **Inclusion Rules** tab shows:

- how many persons in the source had any qualifying drug-exposure entry event,
- how many of those passed the prior-observation window, and
- how many remained after the "no prior diabetes" rule was applied.

If any rule drops your numerator to zero, that is the rule to investigate first. If the diabetes-exclusion rule drops nobody, either there are no diabetes records before metformin in this source (unlikely) or the concept set resolved to nothing (check it in step 3).

3.10 Step 9 — Eyeball some persons

Switch to the **Samples** tab in the same Generation panel. It shows a random subset of the cohort's persons; click any person to open their profile (Chapter 11) with the cohort context set so the membership window is shaded. Skim a few patients: do they look like new metformin users without a prior diabetes record?

3.11 Where to go from here

Variations on this same flow cover most of what ATLAS does:

- Add a second target cohort (e.g. new users of sulfonylureas) and run a *characterization* across both — Chapter 8.
- Define an outcome cohort (e.g. acute kidney injury) and run an *incidence rate* analysis with metformin as the target — Chapter 10.
- Define a small set of event cohorts (insulin, sulfonylurea, GLP-1 agonist, SGLT2 inhibitor) and run a *cohort pathway* analysis from the metformin target — Chapter 9.
- Open one of the cohort's persons in **Profiles** to see the actual records that triggered their cohort entry — Chapter 11.

The reference chapters that follow document each of these features in depth.

PART II

Building Blocks

CHAPTER 4

Data sources

The **Data Sources** area lists every CDM database that the WebAPI is connected to and gives a descriptive view into each. It is the right place to start when you join a new project: before designing a cohort, you want to know what is in the data — how many persons, what condition mix, what time span, and how complete each domain is.

4.1 Picking a source

Open **Data Sources** from the top navigation. The page has two permanent controls:

- a *source picker* dropdown in the page header, showing every CDM database the WebAPI is connected to;
- a left-rail *report sidebar* listing the descriptive reports available for the selected source.

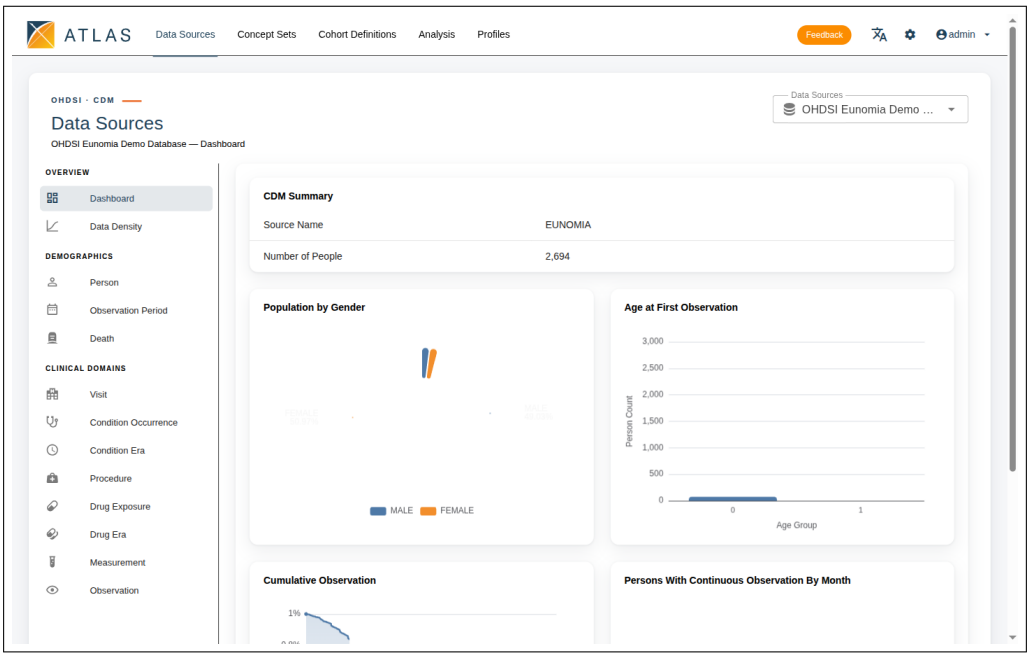


Figure 4.1. Data Sources page. The source-picker dropdown sits in the top-right; the left rail lists report types grouped by Overview, Demographics and Clinical Domains; the main panel renders the selected report.

The dropdown is populated from the WebAPI `/source/sources` endpoint; a source you cannot see may exist but be hidden from your role. Until a source is picked the sidebar is disabled and the main panel shows a "select a data source" hint.

Note

The "source key" — the short text identifier such as `CCAE` or `SYNPUF1K` — appears throughout the URL bar and the JSON of generated cohorts. When two analysts compare results, agreeing on the source key is the unambiguous way to know they are on the same database.

Descriptive reports on a source require that the OHDSI Achilles [9] routine has run against that source's results schema. A source that is configured in WebAPI but has not been Achilles-processed shows up in the list but renders empty report tabs. If your data source page looks bare on every tab, ask your administrator whether Achilles has been run — this is the single most common "set up but nothing shows" cause.

4.2 Reading a source report

Each source exposes the same family of descriptive reports, picked from the left sidebar. The thirteen built-in entries are: **Dashboard**, **Data Density**, **Person**, **Visit**, **Condition Occurrence**, **Condition Era**, **Procedure**, **Drug Exposure**, **Drug Era**, **Measurement**, **Observation**, **Observation Period**, **Death**. A report that has no Achilles results behind it on the chosen source still opens, but renders empty.

4.2.1 Dashboard

The default tab. It summarises the source at a glance: total persons, gender distribution, year-of-birth distribution, observation period density, and a cumulative observation chart. Use it as a sanity check. A source with millions of persons but only a couple of years of follow-up makes a different kind of study possible than a source with hundreds of thousands of persons followed for two decades.

4.2.2 Data Density

A breakdown of how many records exist per domain (condition, drug exposure, procedure, measurement, observation, visit, death) and how that volume changes over calendar time. Sparse rows here usually indicate either a data quality issue or that the source genuinely does not capture that domain.

4.2.3 Person

Demographics in more detail: age at first observation, race, ethnicity, where these are recorded.

4.2.4 Visit

Visit type and visit length distributions. A claims database is dominated by outpatient visits; an inpatient EHR will look very different here.

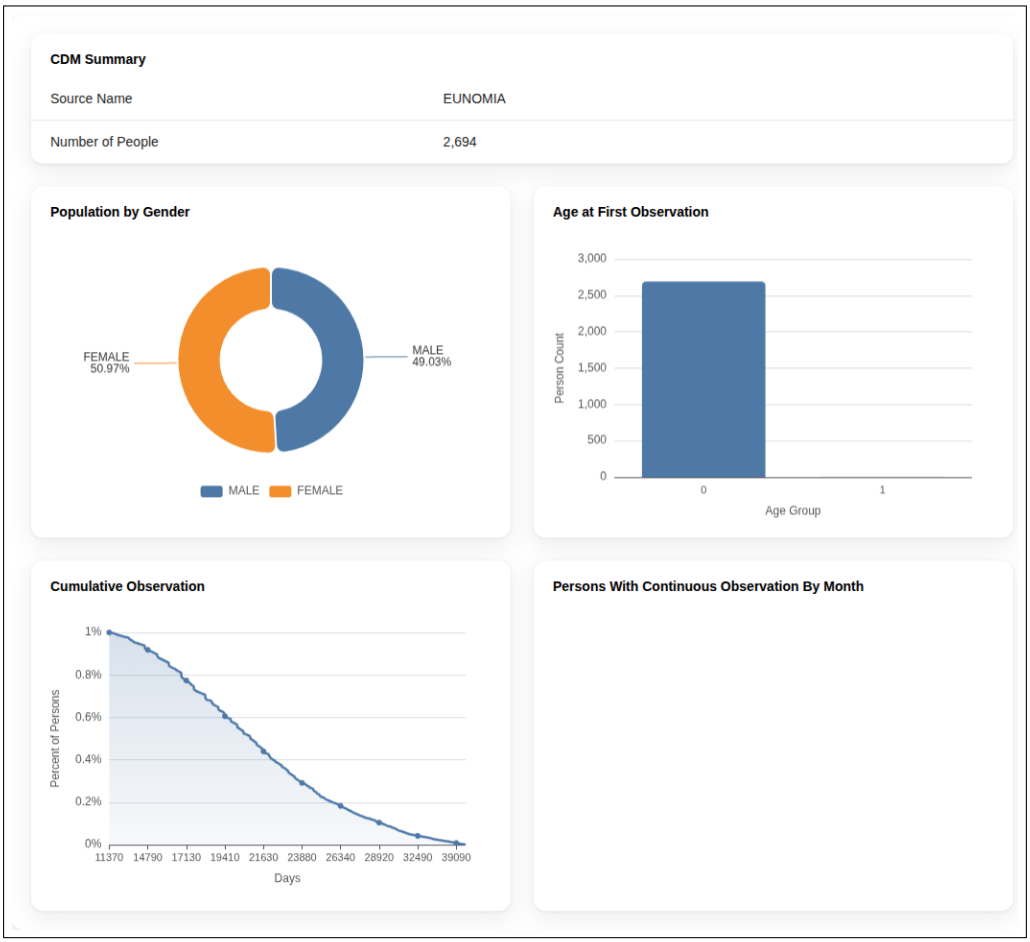


Figure 4.2. Source dashboard: CDM summary, gender split, age at first observation, and the cumulative observation curve.

4.2.5 Condition / Drug / Procedure / Measurement / Observation

One tab per clinical domain, each showing the most prevalent concepts in that domain along with a per-concept drilldown. Scrolling these tabs is the fastest way to learn the "shape" of a source — for example, whether oncology is well represented, whether labs are coded with LOINC, whether inpatient procedures dominate over outpatient.

The default view is a *treemap*: a rectangle filled with nested boxes whose area is proportional to record count, coloured by parent vocabulary class. Hover any tile for the concept name and exact count; click into it to descend into the hierarchy. A toggle next to the treemap switches to a sortable, searchable table view with a concept-name filter — useful when you know the term you are looking for and want to find it directly rather than spotting it visually.

Click any concept row (in either view) to drill into a per-concept report: temporal trend (how the concept's frequency moves over calendar years), gender breakdown, and the distribution of age at first occurrence. The drill-down is the right view when you suspect a concept's counts changed because of a coding-system rollover rather than a true epidemiological shift.

4.2.6 Death

Counts of recorded deaths and their cause-of-death distribution where available. Many ambulatory data sources do not capture death well; that is something to know before designing a survival study.

4.3 Comparing sources

When you have access to several sources you can run the same analysis across all of them; the comparison is what gives observational evidence its strength. From the source list, your usual workflow is:

1. open the dashboard for each candidate source;
2. note the rough size and time span;
3. open the domain tab most relevant to your study (**Drug** for a drug exposure cohort, **Condition** for a disease cohort);
4. check that the concepts you plan to use actually appear with non-trivial counts.

If the prevalence of an entry concept is zero on a source, no cohort generation will ever produce non-empty rows there — even if the SQL runs without error. Catch this in the source report rather than after a long generation.

4.4 Data quality

ATLAS v3.0 itself is not a data-quality tool, but the same Achilles results that drive its descriptive reports also flag the most common data-density anomalies: implausible dates, sudden coding-rate changes, records outside observation periods, and other signals that something upstream of the CDM has shifted. Watch for sharp discontinuities on the temporal trend charts; they usually indicate an ETL change rather than a true epidemiological event.

For a deeper, systematic view, OHDSI ships the *Data Quality Dashboard* (DQD) [14] as a sister tool to ATLAS. DQD runs over 1,500 checks organised by the Kahn data-quality framework [7] (Conformance, Completeness, Plausibility) and produces a per-source dashboard. DQD is not part of ATLAS v3.0; it runs in R against the same CDM and writes its output where your administrator hosts it.

4.5 Patient cache and cohort reports on a source

Some descriptive reports and per-source cohort reports rely on a TrexSQL patient cache being built in advance. Cache management is not exposed on the source page itself; administrators trigger and monitor cache rebuilds from the configuration panel's **Cache** tab (Chapter 14). If a report depends on a missing cache, the report panel itself surfaces the appropriate hint.

The same source detail page is reused later when you open a generated cohort and ask for its reports — see Chapter 7.

CHAPTER 5

Vocabulary search

Every clinical idea expressed in ATLAS v3.0 — a disease, a drug, a procedure — is represented by one or more *concepts* drawn from the OHDSI standardised vocabularies. Vocabulary search is the way you find those concepts. It is also the entry point to building concept sets (Chapter 6).

5.1 Where vocabulary search lives

ATLAS v3.0 surfaces vocabulary search on the **Concepts** page, which has two tabs: **Concept Sets** (the default — a list of saved concept sets) and **Concept Search**. The Concept Search tab is the standalone search experience; the same search is also embedded inline inside the concept-set editor and wherever else a workflow asks you to pick a concept (for example, when you set an entry event in the cohort builder).

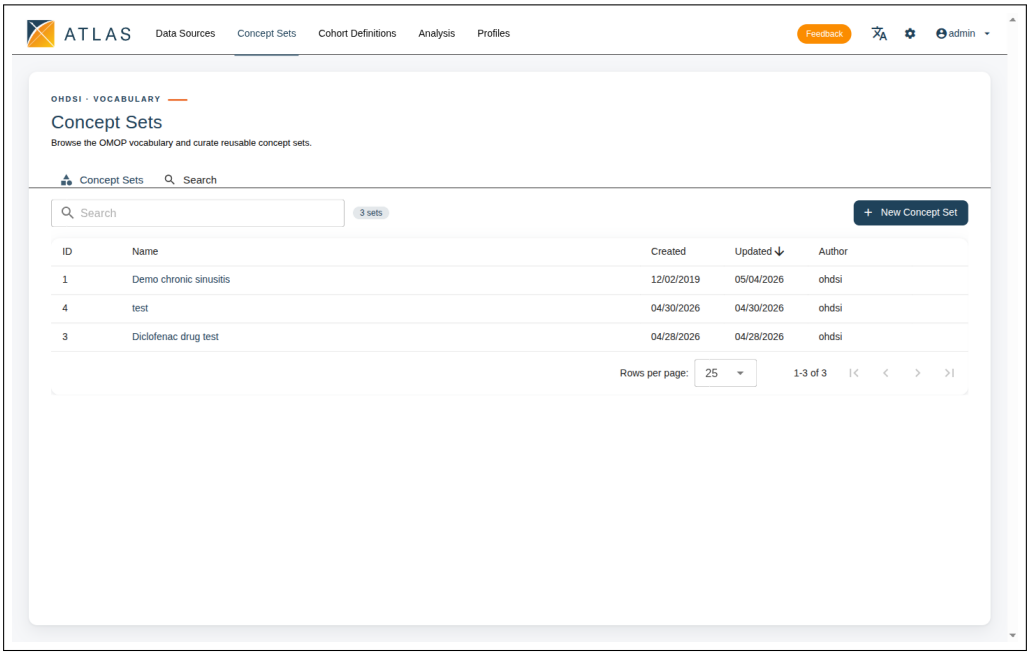


Figure 5.1. Concepts page on the **Concept Sets** tab. The sibling **Search** tab hosts the standalone vocabulary search.

5.2 Running a search

Switch to the **Concept Search** tab. Type a free-text query — a name, a synonym, a code — of at least three characters into the search hero and press `Enter`. The results table fills with matching concepts.

5.2.1 What matches

The search runs against concept names, synonyms, and source codes (ICD-9, ICD-10, SNOMED, RxNorm, LOINC, NDC, and so on, depending on which vocabularies your WebAPI has loaded). A query like `atrial fibrillation` matches the standard SNOMED concept and its descendants; a query like `I48` matches the ICD-10 code and the standard concept it maps to.

Note

The OHDSI vocabulary lifecycle — standard vs. non-standard concepts, what each **Invalid Reason** flag (`D`, `U`, `R`) means, the `Maps to` and `Maps to value` relationships, the role of vocabularies like SNOMED, ICD-10, RxNorm and LOINC — is documented in *The Book of OHDSI*, chapter 5 "Standardized Vocabularies" [20]. This chapter only covers what the ATLAS UI shows.

5.2.2 Reading the result table

The result table has these columns: ID, Code, Name, Vocabulary, Type (the standard-concept flag), Domain, Class, Validity, and four record-count columns abbreviated **RC** (record count for the concept itself), **DRC** (descendant record count), **PC** (person count), and **DPC** (descendant person count). All columns are sortable; there are no per-column filter chips, so narrow a noisy search by refining your query or by sorting on the column that matters most — often **DRC** to surface the most-used concepts in the source.

The record-count columns reflect the data source the page is configured against. If they remain blank, the source has no patient cache built (Chapter 14) or you do not have permission to read it.

5.3 From search to set

Each row in the result table carries a single per-row action:

- **Add** — drop the concept into the concept set currently being edited. The row's button flips to **Remove** once it is in the set.

The per-concept Include / Include descendants / Include mapped / Exclude flags described in older ATLAS releases are not set from the search results page; they are toggled per row inside the concept-set editor itself (Chapter 6).

Note

The exact set of vocabularies you can search is whatever your WebAPI was loaded with from *Athena* [10], the OHDSI vocabulary repository. Two analysts on the same ATLAS v3.0 release but with different vocabulary snapshots can see slightly different concept IDs; record the snapshot date along with your study.

5.4 Search strategies that work in practice

- Start broad, then narrow. Search for the disease by name, pick the highest-level standard concept that still represents what you mean, and rely on *include descendants* rather than listing every variant by hand.
- Cross-check against codes. If you know the ICD-10 chapter for your phenotype, search by code as well; the standard concept you arrive at should be the same.
- Use the source-specific record counts. A concept with zero records in your target source is unlikely to drive cohort entry on it, regardless of how well it represents the disease in the abstract.

CHAPTER 6

Concept sets

A *concept set* is a named, reusable collection of concepts together with rules for how each concept participates. Concept sets are the building blocks of cohort definitions and most analyses; if you find yourself listing the same concepts in two cohorts, you should probably be referencing the same concept set in both.

6.1 Why a list of concepts is not enough

Two ideas separate a concept set from a flat list:

Descendants A standard concept usually has descendants in the OHDSI hierarchy. *Type 2 diabetes mellitus* (SNOMED) has many specific subtypes underneath it. With **Include descendants** ticked on the parent, every subtype is included automatically and stays current as the vocabulary evolves.

Mapped source codes Real patient records are often coded in source vocabularies (ICD-10, RxNorm, NDC). With **Include mapped** ticked, the set captures records that map up to the chosen standard concept even if they were originally recorded under a source code.

A concept set therefore expands at runtime into a — usually larger — list of concept IDs that the cohort SQL actually uses.

6.2 Creating a concept set

1. Open **Concept Sets** from the top navigation. The **Concept Sets** tab is the default; you land directly on the list.
2. Click **New Concept Set**. The editor opens as a side drawer on the right.
3. Give the set a descriptive name in the inline-edit title at the top of the drawer — research groups frequently adopt a convention such as `[Phenotype] [author] [version]`.
4. Switch to the **Search** tab inside the editor and use the vocabulary search (Chapter 5) to find your starting concepts; click **Add** on each row.
5. Switch back to the **Included Concepts** tab to see the concepts in the set and adjust per-row flags.

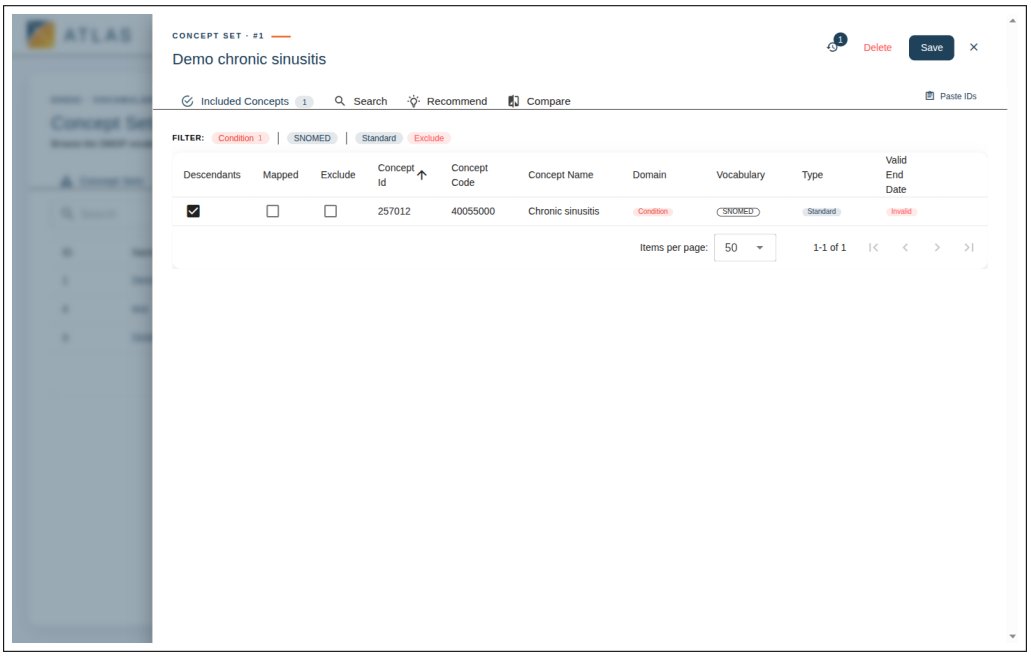


Figure 6.1. Concept-set editor drawer with the **Included Concepts** / **Search** / **Recommend** / **Compare** tabs, the per-row Descendants / Mapped / Exclude flags, and the **Paste IDs** action top-right.

6.3 The Selected tab

The **Included Concepts** tab lists every concept currently in the set, with three per-row toggles and a remove action:

Descendants Whether the set also covers concepts below this one in the OHDSI hierarchy.

Mapped Whether non-standard source codes that map up to this concept count as members.

Exclude When on, members covered by this row are subtracted from the set, even if another row would otherwise include them.

Membership itself is implicit: every row in the table is part of the set unless its **Exclude** flag is on. To take a concept out of the set entirely, use the row's remove action rather than an Include toggle.

These toggles compose. A common shape is "include the parent disease with descendants, then add an exclude row for one specific subtype". The Selected tab also has a filter strip at the top of the table to narrow the displayed rows by standard-concept flag, validity, or exclusion status.

6.4 Bulk-adding concepts: Paste IDs

To the right of the editor's tab strip is a **Paste IDs** button. It opens a dialog where you paste a list of concept IDs (one per line, or space- or comma-separated); ATLAS resolves

them against the vocabulary and reports a matched / unmatched summary. This is the fastest way to import a code list from a spreadsheet or a phenotype paper.

A concept set that resolves to thousands of included concepts is rarely what you wanted. Either narrow the parent concept, turn off descendants, or add specific exclusions until the resolved list reads sensibly.

Before saving a concept set you intend to publish, sit with a domain-aware reviewer in front of the **Included Concepts** tab and the record-count columns. This is the catch step for two common mistakes: (i) a standard concept whose mapped source codes are clinically broader than your phenotype intends (the up-hill mapping problem), and (ii) a descendant subtree that sweeps in obsolete or rarely-used codes a reviewer would have flagged. The complementary check — *orphan codes*, source codes a clinician would have included but the set does not capture — is what the HADES `CohortDiagnostics` package automates [28, 30].

6.5 Recommend

The **Recommend** tab uses PHOEBE 2.0 to suggest additional concepts that are commonly co-included with the ones you have already added. PHOEBE relies on co-occurrence patterns drawn from a large network of OMOP databases, so a concept it suggests is one that other phenotype authors have routinely paired with yours. Review each suggestion before accepting it — the recommender does not know your phenotype's intent, only the company it tends to keep in published code lists.

The tab is unavailable when the WebAPI's vocabulary schema does not include the PHOEBE 2.0 tables; the panel itself surfaces this condition.

6.6 Compare

The **Compare** tab compares the concept set you are editing (**Concept Set 1**) against another saved concept set (**Concept Set 2**). The workflow is:

1. Click **Choose** on the toolbar; pick the second set from the picker dialog.
2. Click **Compare Concept Sets**. ATLAS resolves both sets to concept lists, computes the intersection and the two differences, and renders a Venn diagram alongside a result table.
3. The result table has a **Match** column that tags every concept as *Both*, *Only in set 1*, or *Only in set 2*, plus the usual concept ID, code, name, domain, vocabulary and class columns. Sort or scroll the *Only in* rows to read off exactly which concepts moved when you compare two phenotype iterations.

4. **Export** on the toolbar writes the result table to a file for review or attachment to a phenotype submission.

Use the comparison to:

- review what changed between two phenotype iterations before retiring the older one — the *Only in set 1* rows are what the older version captured that the new one no longer does;
- check whether two phenotypes that should be disjoint (e.g. a positive and a negative control) actually are — the Venn diagram should have an empty intersection;
- migrate from a peer-curated library set to a local one without losing terms accidentally.

A worked iteration: you have set v1 published in a study, set v2 in development. Open v2, switch to **Compare**, choose v1 as **Concept Set 2**. The *Only in set 1* rows are the terms v2 dropped — defend each one in the change log. The *Only in set 2* rows are the new terms in v2 — justify each. The *Both* rows are the unchanged backbone. Do this review before you point any cohort at v2.

6.7 Saving

Save (or **Create** on a new set) writes the current concept set to WebAPI. The first save creates a permanent ID; later saves create new versions (Chapter 13). **Delete** removes the set; both buttons are disabled when you do not hold the relevant per-entity permission.

Include Mapped pulls in records whose *source* code maps to your standard concept, even when the mapping is "up-hill" to a broader parent than the source code itself denotes. For precision-sensitive phenotypes, leave it off and rely on standard concepts the source database was ETL'd to use directly. The mapping mechanics are covered in *The Book of OHDSI*, chapter 5 [20].

6.8 Starting from a phenotype library

Before authoring a concept set from scratch, check whether the phenotype already exists in a community library. Several peer-curated libraries can be searched first:

- *The OHDSI Phenotype Library* [17] holds peer-reviewable cohort definitions in OMOP-CDM ATLAS JSON. Each entry has a stable identifier and each release of the library carries its own DOI for citation.
- *The HDR UK Phenotype Library* [3] indexes more than a thousand phenotype algorithms expressed as clinical code lists — ICD, SNOMED, Read — across

cardiology, oncology, COVID-19 and other areas, contributed by groups across UK research; it is API-accessible and ships an R client.

- *EHDEN* [2] (the European Health Data & Evidence Network) curates phenotypes used in pharmacoepidemiology studies on its federated 100M-patient OMOP estate, and produces training material on phenotype definition, characterisation, and evaluation that complements the OHDSI tools.

A concept set or cohort imported from any of these is almost always cheaper to adapt than to author — and visibly more defensible at peer review — than starting with a blank expression.

When you reuse a definition from the OHDSI Phenotype Library, record both the cohort identifier and the library release version (each release has its own DOI). Cohort definitions accepted into a release are immutable for that release, but a definition can be marked deprecated [D] or in error [E] in a later one. Pinning the release insulates your study from those later changes.

6.9 Discovery

Concept sets accumulate quickly. The list page on the **Concept Sets** tab carries a single free-text filter at the top that matches against the set name, plus sortable columns including **Author**. Adopt a naming convention — e.g. `[Phenotype] [author] [version]` — so the free-text filter remains useful as the library grows.

PART III

Cohort Definitions

CHAPTER 7

Cohort definitions

A *cohort* is a set of persons who satisfy one or more criteria for a duration of time. A *cohort definition* is the formal, machine-executable description of those criteria. This chapter is the longest in the manual because the cohort builder is where you spend most of your time in ATLAS v3.0.

The cohort builder is divided into numbered sections matching the logical pieces of a definition — entry event, inclusion criteria, exit, and censoring — plus a toolbar that opens generation as a side panel.

Note

A cohort definition built in ATLAS is one stage of a longer phenotype-development workflow that includes diagnostics, evaluation, and submission to a community library. The methodology is documented in *The Book of OHDSI* [20] (chapters 10, 16, 18) and is out of scope for this manual; we focus on the ATLAS controls.

ATLAS's cohort model lets one person belong to the same cohort during *multiple non-overlapping time periods* — a person who initiated metformin in 2014 and again in 2019 has two cohort entries. They are never in the cohort twice simultaneously. This matters when you read reports: the "cohort entries" count is greater than or equal to the "persons" count, and the difference tells you how often re-entry happens in your data.

If you came from SAS or R, you are looking for `NOT EXISTS`. ATLAS does not have it. Every "absence" constraint — "no prior exposure to drug X", "no diagnosis of Y" — is expressed as an inclusion rule with cardinality *exactly 0 occurrences* in the relevant temporal window. This pattern recurs throughout the chapter; the cohort builder itself has no negation operator.

7.1 Live patient counts

The most prominent change from ATLAS 2.15 is the *patient-count bar* pinned to the top of the cohort builder. When the deployment has TrexSQL enabled and a per-source patient cache built (Chapter 14, "Caches and TrexSQL"), the bar shows a running

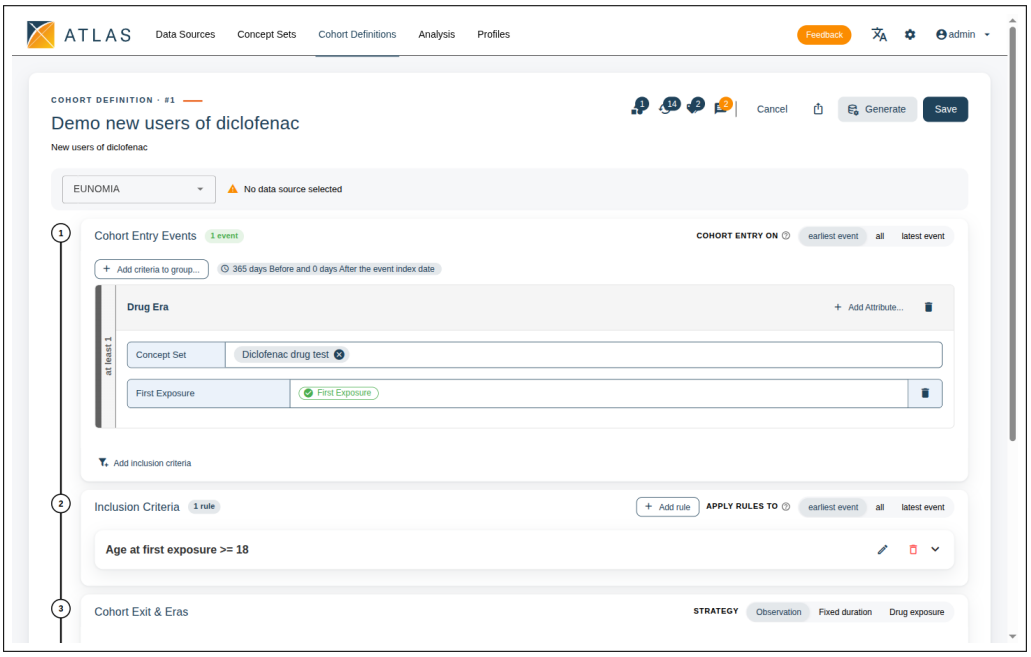


Figure 7.1. The cohort builder. A numbered step rail walks down the page (Cohort Entry Events, Inclusion Criteria, Cohort Exit & Eras); the toolbar carries Cancel, Generate, and Save.

estimate of the cohort size as you edit — no **Generate** round-trip required. The bar carries three controls:

- a *dataset picker* on the left: pick the source the live count runs against. Each source is tagged with its cache state in the dropdown (**Ready**, **Building**, **Stale**, **Error**, or **Not Built**).
- a horizontal *progress bar* sized to the cohort-over-total ratio.
- a *count display* reading ⟨`cohort`⟩ / ⟨`total`⟩ `patients`.

What the bar shows depends on the cache state:

Cache Ready. You see a live cohort count that re-evaluates after every edit. A small clock-with-alert icon appears if the cache is **Stale** (older than the underlying data); hover for the last-built timestamp.

Not Built or Building. The bar shows an amber warning ("Cache is not ready") and no count. Ask an administrator to build the cache from the configuration panel's **Cache** tab.

Error. The bar shows the error message and a **Retry** button.

Empty cohort expression. The bar reads "Add criteria to see patient count" until you add an entry event.

A rule that drops the live count from 80% to 0% the moment you tick it is almost always a misconfigured concept set or an inverted cardinality, not a real epidemiological signal. Treat the bar as your in-place sanity check: it catches mistakes that would otherwise only surface after a full **Generate**.

Note

The bar is hidden entirely on deployments where TrexSQL is not enabled. If you do not see it at the top of the cohort builder, the deployment runs without it — proceed with **Generate** as the only feedback loop.

7.2 Listing and creating cohorts

Open **Cohort Definitions** from the navigation. The list page supports two views — tile and table — toggled in the top-right. Cards show the cohort name, author, last modification date, tags, and quick actions. Use the search bar to filter by name; sort by recency to find what you were last working on.

- **New Cohort** starts an empty definition.
- **Import** accepts a cohort definition JSON file (the format ATLAS writes when you export). ATLAS v3.0 will run it through its ATLAS-2.x converter where needed before the editor opens.
- Clicking a card opens that cohort in the editor.

7.3 The four logical pieces of a definition

Every cohort definition answers four questions, in this order:

1. **Who enters the cohort, and on what date?** — entry event (also called primary criteria, or initial event).
2. **Of those entrants, who do we keep?** — inclusion criteria with cardinality and temporal logic.
3. **When does the cohort end for each person?** — exit criteria.
4. **What ends membership early?** — censoring events.

The rest of this chapter walks each in turn.

7.4 Entry event (primary criteria)

The entry event defines what kind of clinical record makes a person a candidate for the cohort, and which date of that record becomes the person's *index date*. You add at least one entry event from one of these criterion types:

- Condition occurrence, Condition era

- Drug exposure, Drug era, Dose era
- Procedure occurrence
- Measurement
- Observation, Observation period
- Visit occurrence, Visit detail
- Device exposure
- Death
- Specimen
- Payer plan period
- Demographic, Location region

When you add an entry event, the editor presents a panel with a concept set picker followed by a list of *attribute filters* appropriate to the domain. For a condition occurrence, those attributes include age at occurrence, gender, condition type, condition status, and visit type. For a drug exposure, they include dose, quantity, days supply, refills, route, and stop reason.

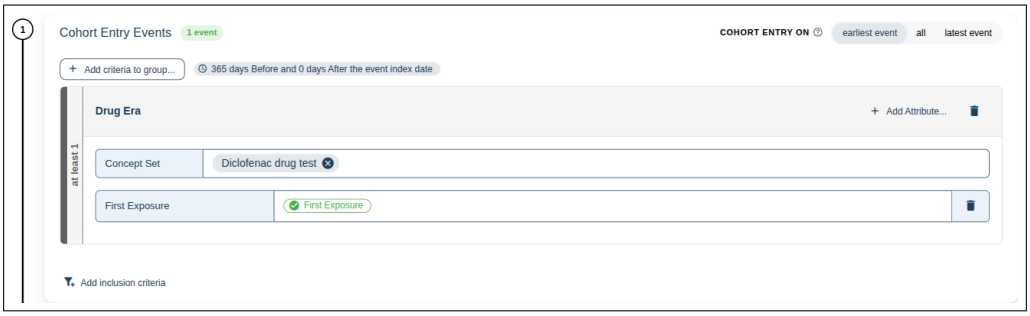


Figure 7.2. Entry-event panel. The **Cohort entry on** toggle on the right (earliest event / all / latest event) is the per-section qualifying-events limit; the observation-period chip ("365 days Before and 0 days After the event index date") is the look-back / follow-up requirement.

7.4.1 Cohort entry observation window

Below the entry event you set how many days of continuous observation you require before and after the index date.

- *Days before index* controls washout. A value of 365 means "the person must have been observable in the data for at least a year before they entered the cohort", which is the usual way to require a clean look-back period.
- *Days after index* ensures a minimum follow-up.

Encode the required look-back as *Days before index* on the entry event itself, not as an inclusion rule. Doing it as an inclusion rule biases downstream incidence and characterization denominators; the OHDSI Themis convention is the entry-event form [20, 25].

7.4.2 Three qualifying-event limits

ATLAS exposes three independent limits that decide *which* qualifying events count, each as an **earliest event / all / latest event** toggle:

Cohort entry on applies to the entry-event panel itself. **earliest event** produces a *new-user* or *incident* cohort (only the person's first qualifying record); **all** lets every qualifying record create a cohort entry; **latest event** keeps only the most recent.

with applies to the optional **Add inclusion criteria** block that further restricts entry events *before* the inclusion rules below run. Same three values; controls which qualifying entry the additional criteria are evaluated against.

Apply rules to sits above the inclusion-rule list and decides whether the rules below apply to the first, every, or last qualifying record per person. With **earliest event**, ATLAS narrows to the earliest qualifying entry and applies the inclusion rules to that one event — usually what new-user designs want.

all plus generous descendants on a chronic-disease concept set is a fast way to multiply rows by an order of magnitude. Pick the limit on each section that matches the question.

7.5 Inclusion criteria

Inclusion criteria are applied after entry events. Each rule is itself a small cohort criterion — "at least one diagnosis of X in the 365 days before index", "no exposure to Y any time before index" — and the person must satisfy every rule that you mark as required.

Note

Each inclusion rule you add raises specificity at the cost of sensitivity — the trade-off is the same as in any phenotype algorithm. Inspect the inclusion-rule report and a sample of profiles (Chapter 11) to see which side of that trade you have landed on. The methodology is covered in *The Book of OHDSI* [20], chapter 10.

7.5.1 Anatomy of a rule

1. **Domain.** Pick what kind of event the rule is about (condition, drug, procedure, measurement, observation, visit, device, era, etc.).

2. **Concept set.** Reference an existing concept set or open a popup to build one inline.
3. **Attribute filters.** Same domain-specific list as for the entry event.
4. **Cardinality.** Choose *at least*, *at most*, *exactly*, plus a count N.
5. **Temporal window.** Specify the time relative to index in which the events count, e.g. *Days: 365 days before to 0 days before*, with *Use index date* or *Use index + cohort exit* as anchors.

7.5.2 Grouping criteria with AND / OR

Each inclusion rule can hold more than one criterion. The **+Add criteria to group** button under a rule adds another criterion to the same group; a dropdown at the top of the rule chooses how the criteria combine:

All criteria (logical AND) — the person must satisfy every criterion in the group.

Any criteria (logical OR) — the person must satisfy at least one.

At least N of — the person must satisfy any N of the criteria, where N is configurable.

A rule can also contain a nested *sub-group* with its own combinator, which is how complex phenotypes like "diabetes diagnosis AND (insulin use OR HbA1c $\geq$ 6.5)" are expressed. Build the nesting carefully — two criteria at the same level combine differently from one criterion plus a sub-group.

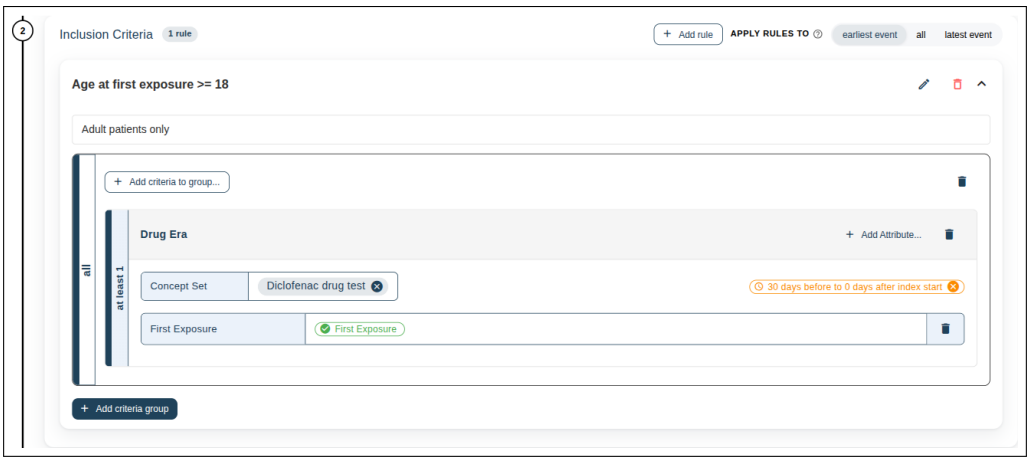


Figure 7.3. Expanded inclusion rule. Inside the rule, criteria groups carry a cardinality ("at least 1") and each criterion carries a concept set, attribute filters, and a temporal-window chip ("30 days before to 0 days after index start").

A phenotype that requires a confirming second event after index introduces *immortal time* between the first and second records. Either index on the second event in ATLAS (so the immortal window precedes index) or keep the confirming window short. The bias is documented in Suissa [36] and Swerdel et al. [39].

7.5.3 Reading the inclusion report

After generation, the **Inclusion Rules** tab inside the Generation side panel shows, for each rule, how many candidates remained after applying it in turn. This drop-off chart is one of the most useful things in ATLAS v3.0: it tells you which criterion is doing the work and which is silently keeping or dropping everyone.

When counts surprise you, read the chart as a sequence of suspects:

- A rule that drops to near-zero on its own usually means the concept set resolved to nothing on the source — check the concept set's **Included Concepts** tab and re-run on a source with known coverage.
- A rule that drops nobody is either trivially satisfied (the temporal window is too wide, the cardinality is too lax) or redundant with an earlier rule. Either way it is not pulling weight.
- A rule that drops disproportionately many people compared with a peer source signals an ETL or vocabulary-mapping difference — the same rule against the same JSON should drop comparable fractions. The HADES `CohortDiagnostics` package formalises this cross-source comparison with attrition tables side by side [30].

7.6 Cohort exit and censoring

7.6.1 Exit criteria

Exit defines the per-person end date. Choose one of:

End of continuous observation. The person leaves the cohort when they leave observation in the source. The most common choice for prevalent or chronic-disease cohorts.

Fixed duration. A specified number of days after index. Useful for time-to-event analyses with a hard horizon.

End of continuous drug exposure. Used for drug-utilisation cohorts. You configure the persistence window (the gap between consecutive exposures still considered the same era) and the surveillance window.

Event criteria. Exit on the first occurrence of a specified event after index.

7.6.2 Censoring events

Censoring events end the person's membership early when an outcome makes continued follow-up nonsensical. Common censoring choices include death, end of insurance enrolment, and onset of an outcome you want to exclude.

7.6.3 Era collapse

For cohorts where the same person can enter and re-enter, you can **Collapse cohort entries into eras** with a configurable gap days. With a 30-day gap, two cohort entries

20 days apart are merged into one continuous era; with a zero gap, every entry stays separate.

The 30-day persistence window is the OHDSI convention for drug eras because it has historically performed well for drug–outcome estimation, not because it is biologically motivated. For long-acting injectables and depot formulations, 90 days or more is common; for acute treatments where a real treatment break is clinically meaningful within days, shorter values matter. Treat the window as a study parameter: report it in the manuscript and, on sensitive analyses, run the cohort with ± 30 days around your choice as a sensitivity check.

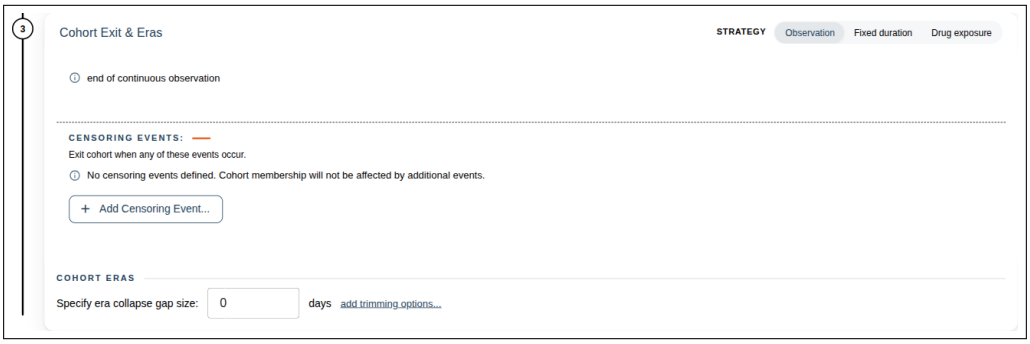


Figure 7.4. Cohort Exit & Eras. The strategy toggle picks Observation, Fixed duration, or Drug exposure; Censoring Events end membership early; the era collapse gap (in days) merges adjacent cohort entries.

7.7 Concept sets used by this cohort

The **Concept Sets** tab inside a cohort definition lists every concept set the definition references. From here you can review the resolved included concepts without leaving the editor. Renaming a concept set here renames its inline copy on this cohort but does not affect the shared library version — that is handled through the version history (Chapter 13).

Note

A cohort definition stores its own *copy* of every concept set it references. This copy travels with the cohort JSON when you export it, so a downstream user can reproduce the cohort even without access to your concept-set library. The cost is that updating a concept set in the library does *not* retroactively update the embedded copy on every cohort that uses it — re-saving the cohort against the library version is a deliberate, explicit action.

7.8 Generation

Generation runs the definition's SQL against a chosen source. Click **Generate** on the cohort toolbar to open the **Generation** side panel:

1. The left rail shows a tile per connected source. Click a tile to start generation on that source — WebAPI compiles the SQL and executes it, and the tile shows progress.
2. When a source finishes, the tile shows person and record counts plus the time it took.
3. Click into a finished source to see two report tabs on the right: **Inclusion Rules** (the per-rule attrition chart described in the previous section) and **Samples** (a random subset of the cohort's persons, covered below).

7.8.1 Cohort samples

The **Samples** tab inside the Generation panel is the fastest path from a generated cohort to individual patient review. Click **New sample** to open the sample-creation dialog; specify:

Sample name a descriptive label, e.g. `review-2025-01-women-over-65`.

Number of persons how many to draw, up to the per-source maximum.

Gender (optional) one or more of **Male**, **Female**, **Other / non-binary**.

Age (optional) a comparator (*less than*, *greater than*, *between*, etc.) plus a value or range.

ATLAS runs a server-side random draw constrained by the filters and shows the result as a list of person IDs. Click any person to open their profile (Chapter 11) with the cohort context set, so the cohort-membership window is shaded on the timeline. **Refresh** re-rolls the same sample with a new draw, **Delete** removes it.

Use a sample of 20–50 patients as the lightweight half of phenotype validation: skim each profile, note any obvious mismatches, and adjust the cohort definition. This is much cheaper than a full PheValuator run (Chapter 11, "Going further") and catches the gross errors first.

Note

Generation is the only step that actually queries patient data, and on large sources it can take a while. It is also the first place a flaw in the definition — a concept set that returns nothing, a temporal window that excludes everyone — becomes visible. The inclusion-rules report inside the Generation panel is your first stop when counts surprise you.

7.9 Going further: external diagnostic tools

The Generation panel's reports are deliberately a quick view — attrition by rule, plus a sample. For the deeper diagnostics expected of a sharable phenotype, two external HADES R packages consume the JSON ATLAS exports:

CohortDiagnostics [28, 30] produces cross-source attrition, index-event breakdown, source-code review, orphan codes, incidence by age / sex / year, cohort overlap, and time distributions. Fast, runs without a gold standard.

PheValuator (Chapter 11, Section 11.4) estimates sensitivity, specificity, and PPV against a probabilistic gold standard.

Both are recommended evidence for OHDSI Phenotype Library submissions [26]. The methodology behind them lives in *The Book of OHDSI* [20], chapters 16 and 18.

For *negative-control outcomes* — the supplementary cohorts many OHDSI designs use to detect residual bias — ATLAS is just where you build them. Tag them `negative-control`; the methodology and recommended counts are covered in the Book of OHDSI's Method Validity chapter.

7.10 Saving, copying and exporting

Save the cohort with a meaningful name as soon as you start editing, even before any criteria are filled in. The first **Save** creates a permanent ID and unlocks version history; subsequent edits then accumulate as versions you can roll back to, rather than as one giant undo stack you can lose.

- **Save** writes the cohort to WebAPI. Repeated saves create new versions (Chapter 13). The button is disabled when you do not hold write permission on the cohort.
- The **Export** action on the toolbar opens a small menu with two options: **Download as file** writes the definition's JSON to disk, and **Copy to clipboard** copies the same JSON for pasting elsewhere. The same JSON can be re-imported into this ATLAS instance or another.

7.10.1 Tagging a cohort

The cohort toolbar carries a tag icon (MDI-TAG-MULTIPLE) with a count badge of the current tags. Click it to open the **Manage Tags** dialog, which lists every tag defined in the deployment grouped by tag group; tick the ones that apply.

Tags are deployment-wide artefacts — they are created and curated from the configuration panel's **Tags** tab (Chapter 14), not from the cohort. A useful convention is one tag per project area (e.g. `cardiology`, `oncology`), one per study phase (`exploratory`, `published`, `negative-control`), and a small set of reusable qualifiers (`phenotype`, `drug-class`, `outcome`). Tags appear on cohort cards in the list page and are searchable by sorting on the tag column.

The same tag mechanism applies to concept sets (Chapter 6), characterizations (Chapter 8), pathways (Chapter 9), and incidence rates (Chapter 10); the icon, dialog, and behaviour are identical across editors.

7.10.2 Auto-save and draft recovery

ATLAS auto-saves a draft of the cohort to the browser's session storage every 30 seconds while the editor is dirty, even before the first deliberate **Save**. The mechanism matters in three situations:

- **Token expiry mid-edit.** If your session expires and you log back in, the draft is restored on return to the editor.
- **Accidental tab close.** Closing the tab and reopening it within the same browser session restores the draft.
- **New cohort that was never saved.** The draft is keyed to the editor, not to a cohort ID, so an unsaved new cohort survives a refresh.

The draft is *not* synced to other browsers, devices, or tabs (it is per-tab), and it is cleared when the cohort is deliberately saved or the browser session ends. Treat it as a crash-recovery net, not a substitute for **Save**.

PART IV

Analyses

CHAPTER 8

Characterizations

A *characterization* summarises the clinical features of one or more cohorts. Where a cohort report (Chapter 7) describes one cohort on one source, a characterization is broader: it can put several target cohorts side by side, attach outcomes, and apply a deliberate, reusable set of feature analyses.

8.1 What you can answer with a characterization

Typical questions:

- Among new users of drug A versus drug B, what does the patient mix look like at index — age, comorbidities, prior medications?
- In a population with disease X, how does the burden of related conditions evolve in the year after diagnosis?
- Across five different data sources, how comparable are the baseline characteristics of the same target cohort?

A well-designed characterization is what you publish alongside a study to demonstrate that "the patients in my cohort look like what you would expect of patients with this condition".

Note

For comparative-effectiveness work, you typically run a characterization with two target cohorts (the *target* and the *comparator* of an active-comparator new-user design [42]) before propensity adjustment. The study design and methodology are covered in *The Book of OHDSI* [20], chapters 7 and 12.

Note

The ATLAS characterization presets give four standard time-window durations relative to the cohort entry (*index*) date: *Any time prior*, *Long-term* (365 days), *Medium-term* (180 days), and *Short-term* (30 days). A typical baseline bundle covers all four so reviewers can see how prevalence changes as the look-back window narrows. The underlying distinction between baseline, post-index, and time-at-risk windows is methodology covered in *The Book of OHDSI* [20], chapter 11.

8.2 Anatomy of a characterization

Open Analysis › Characterizations and click **New Characterization**. The editor has three sections, in order:

Cohorts. One or more cohorts whose members you want to describe. There is no separate outcome-cohort slot — outcome-stratified characterisations are expressed by adding the outcome cohort here and a matching subgroup below.

Feature analyses. The list of covariates to compute. Each feature analysis is itself a reusable artefact (Chapter 12). A typical default set includes demographics, conditions in the year before index, drug exposures in the year before index, prior visits, and Charlson comorbidity components.

Subgroup analyses (optional). Overlapping selections of the population built like miniature inclusion rules: concept set, attribute filters, cardinality, and a temporal window relative to index. Each subgroup becomes an extra column in the result alongside the main cohort. Use them to ask questions like "what does the cohort look like *among those with prior MI?*" without having to redefine and regenerate the cohort itself. A **Calculate subgroup analyses only** toggle is available when you want subgroup columns without the full characterisation.

The data sources to run on are picked at generation time, not on the design form.

8.2.1 Building a subgroup

New subgroup opens a small editor that mirrors the inclusion-rule panel from the cohort builder: name the subgroup, reference or build a concept set, set attribute filters and a cardinality, and specify a temporal window relative to index. The subgroup membership is computed at characterization time and yields a column whose values are restricted to persons satisfying the criteria.

8.3 Generating and viewing results

The canvas shows a *data-source run table* at the top with one row per connected source. Each row carries the source name, its current status (**COMPLETED**, **RUNNING**, **FAILED**, or none if it has never been run), the last-run timestamp, duration, and per-row actions: **Run**, **Cancel** (while running), and **Show history** with a count badge. WebAPI executes one characterization run per (cohort × source) pair; for each cohort it computes every feature analysis.

Click **Show history** on a source's row to open the **Previous runs** dialog scoped to that source — it lists every historical run with its date, status, duration, and an eye icon to reopen completed results. The same dialog opens from the matching action on the IR and pathway editors, so the navigation pattern is identical across the three analysis types.

When a run completes you see two output styles:

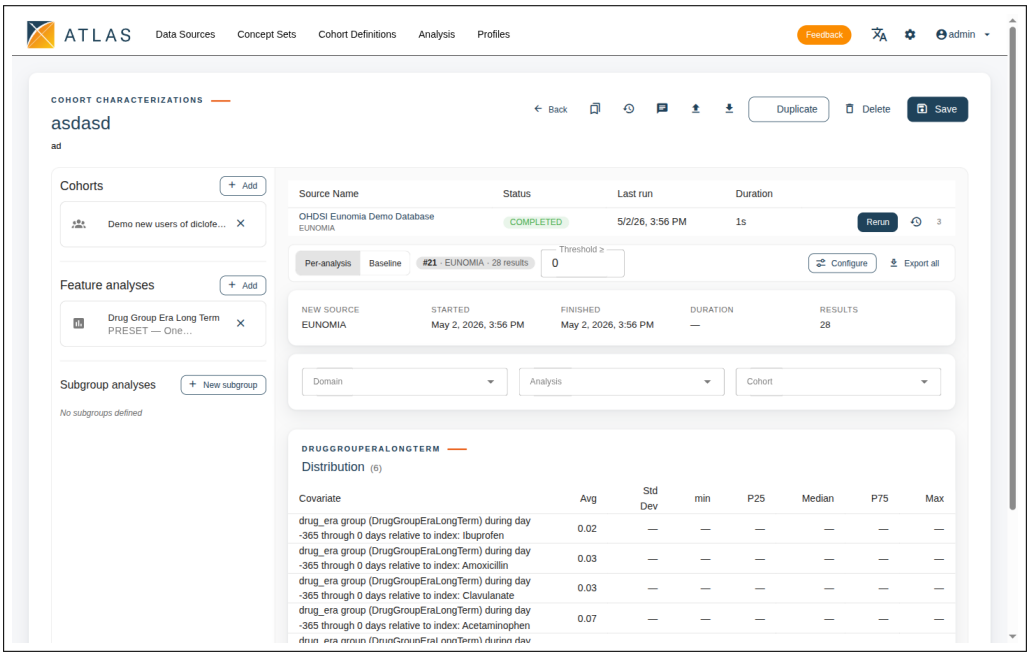


Figure 8.1. Characterization editor. The design rail on the left lists Cohorts, Feature analyses, and Subgroup analyses; the canvas on the right shows the data-source run table above the result panels.

Prevalence-style features (binary covariates such as "had diabetes in the prior year"): a percentage with a count column, sorted by prevalence within each target cohort.

Distribution-style features (continuous covariates such as age): mean, standard deviation, median, and quartiles.

A filter strip above the result table carries three multi-selects — **Domain**, **Analysis**, and **Cohort** — that you combine to narrow a busy result to the rows relevant to the question at hand. Each row has an **Explore** action that opens a concept-hierarchy drill-down — click into a high-level row like "Cardiovascular condition — prior" to see which specific concepts contributed to it.

8.4 Export

Export CSV on the canvas toolbar dumps the entire result table for use in a spreadsheet, a manuscript table, or further analysis in R. The downloaded file is named `characterization-{executionId}.csv` so you can match it back to a specific past run.

8.5 Common pitfalls

- **Empty rows on a source.** If the target cohort generates zero persons there, every covariate is necessarily blank. Check the cohort generation log first.
- **Picking too many features.** Each feature analysis costs time and produces a row in the report. A scoped, reproducible set is more useful than an unfiltered dump.

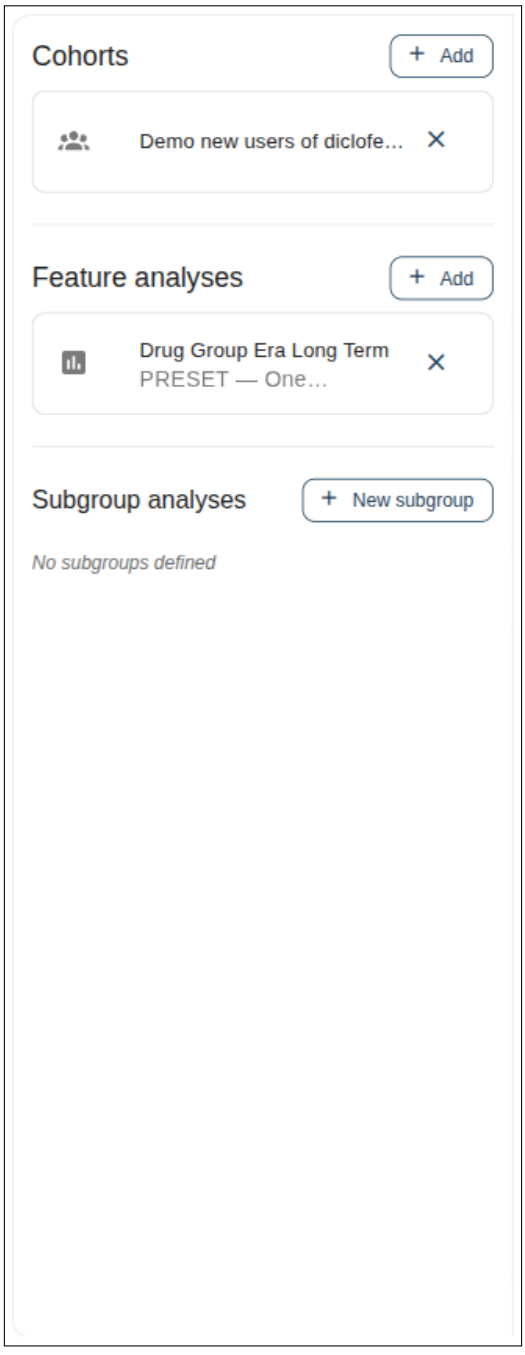


Figure 8.2. The design rail in detail: a Cohorts section, a Feature analyses section listing each linked feature with its type and stat type, and a Subgroup analyses section with the **New subgroup** action.

- **Window mismatch.** Feature analyses define their own windows relative to index. If your cohort exit is shorter than a covariate's window, the covariate may pull data from outside the person's cohort membership.

8.6 Reproducing a characterization in R

Once a characterization is saved, the same definition can be re-executed in R against a CDM the WebAPI may not be connected to. The HADES `FeatureExtraction` [15] package consumes the same JSON the ATLAS v3.0 editor produces; build a `covariateSettings` object via `createCovariateSettings(...)` or, for the aggregated form ATLAS computes, `createDefaultCovariateSettings(...)` with `aggregated = TRUE`. The R workflow is the right choice when the result needs to feed a downstream HADES analysis or when you want to add custom covariates not expressible in the ATLAS UI.

CHAPTER 9

Cohort pathways

Cohort pathways answer the question: in what order do clinical events happen for the people in my cohort? You feed in a target cohort plus a small set of *event cohorts* (each representing a treatment or a clinical milestone), and ATLAS v3.0 produces a sunburst-style diagram that visualises the most common sequences. The technique was used at network scale to characterize treatment patterns across the OHDSI collaborators in Hripcsak et al. [5].

9.1 When this analysis is the right tool

Pathways shine when the comparison of interest is sequential rather than cross-sectional. Examples:

- Among newly diagnosed type 2 diabetes patients, what is the usual order in which oral antidiabetic drug classes are tried?
- For patients with depression, how often is the second prescribed medication a different class from the first?
- In oncology, what does the line of therapy distribution look like?

9.2 Setting up a pathway

Open Analysis › Cohort Pathways, click **New Pathway**, and fill in:

Target cohorts. One or more cohorts whose pathways you want to compare. The result page lays out one sunburst per target.

Event cohorts. Each event cohort represents one node type in the pathway — typically one drug class or one milestone per cohort. The shorter and more orthogonal this list, the more readable the resulting diagram.

Collapse window (days). Two event cohorts that overlap inside this window are treated as a combination at the same step rather than as a sequence. Pickable values: 1, 3, 5, 7, 10, 14, 30.

Minimum cell count. Sequences with fewer than this many persons are suppressed (selectable from 1 to 10). Pair this with whatever small-cell suppression policy your project or IRB requires so the diagram never reveals near-unique patient trajectories.

Maximum path length. How many steps the analysis follows past the index event, from 1 to 7. Longer paths yield more detail but produce sunbursts that fragment into increasingly small wedges and rapidly cross the minimum cell count threshold.

Allow repeats. When on, the same event cohort may appear more than once on a single person's pathway (e.g. a drug re-initiation after a gap). When off, the second occurrence is collapsed into the first.

Targets are typically *treatment-initiator* cohorts (e.g. "new users of any first-line antihypertensive"), and event cohorts are *one per intervention* (one per drug class, one per procedure type) [5]. Keeping the event cohorts orthogonal and small in number is the single biggest determinant of whether the resulting diagram is readable.

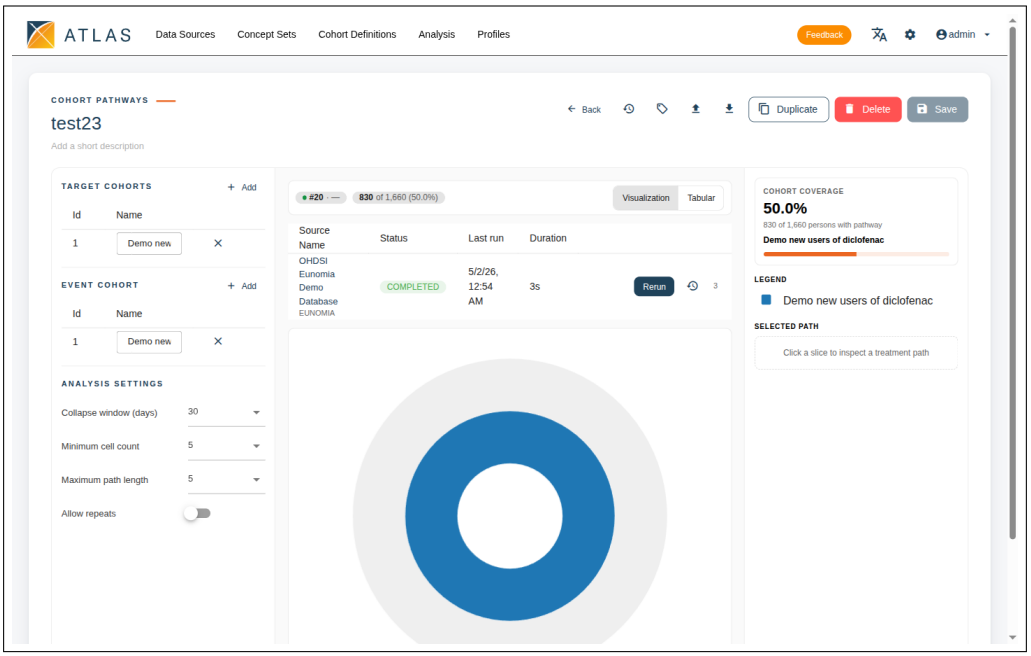


Figure 9.1. Cohort-pathway editor. The design rail on the left holds Target cohorts, Event cohort, and the Analysis settings (Collapse window, Minimum cell count, Maximum path length, Allow repeats). The canvas in the centre shows the data-source run table above the sunburst; the cohort coverage and legend live in the right rail.

9.3 Reading the result

The sunburst shows the index event at the centre and each subsequent step as a ring expanding outwards. The wedges in each ring are colour-coded by event cohort, drawn from a 20-colour palette — the legend table beside the chart maps each colour to the event cohort it represents, and the same colour appears at every step that event takes part in. The angular extent of every wedge is proportional to the number of persons whose pathway took that sequence.

Hovering a wedge shows the count at that step. Clicking a wedge opens the *path details* table beside the diagram, which lists every person-pathway containing that node, with columns for **Event**, **Remain**, **%**, **Diff**, and **%** — useful when you want to read off a

specific path verbatim rather than guess from the geometry. The chart can be toggled to a table-only view from the canvas toolbar.

Generations are dispatched per source from the data-source run table at the top of the canvas: each row's **Run** action queues a generation against that source, and the row updates with status, last-run timestamp and duration. **Show history** on a row opens the **Previous runs** dialog scoped to that source, where you can reopen any historical diagram — the same dialog that ships with characterizations and incidence rates.

A diagram that looks visually busy but where the largest sequences only account for a few percent of the cohort suggests the event cohorts are too granular. Consolidate related medications into class-level event cohorts and re-run.

CHAPTER 10

Incidence rates

The *incidence rate* module estimates how often new outcomes occur inside a target population per unit of person-time. For each (target cohort × outcome cohort × source) combination, ATLAS v3.0 returns persons-at-risk, time-at-risk, outcome cases, and the rate expressed per a chosen multiplier (commonly 1,000 person-years).

Note

The formal definitions of *incidence proportion* (per 1,000 persons) and *incidence rate* (per 1,000 person-years), and the "first event only, prior cases excluded" semantics ATLAS uses, are covered in *The Book of OHDSI*, chapter 11 [20].

10.1 Setting up an incidence rate analysis

Open Analysis › Incidence Rates and click **New Incidence Rate**. Inputs:

Target cohorts. The populations whose outcome incidence you want to estimate.

Outcome cohorts. One per outcome you are studying.

Time at risk. Set start and end as anchor + offset pairs: each side picks an anchor date (**Cohort start** or **Cohort end**) and a number of days to offset. So an intent-to-treat 365-day window is *Start: Cohort start + 0 days*, *End: Cohort start + 365 days*; a per-protocol window pegs to **Cohort end**. Time-at-risk is automatically clipped at end of observation and at the first outcome.

Study window (optional). Restrict the analysis to a calendar date range with explicit **Start date** and **End date** fields. Use it for cohorts that should be analysed within a specific reporting period.

Stratify rules (optional). User-defined strata, each one a name plus a small criterion expression that mirrors the cohort-builder inclusion rules (concept set, attribute filters, temporal window). Add a rule per age band, per gender, per calendar year, or per custom partition; the result table breaks the rate down by every rule that evaluates to true for a person.

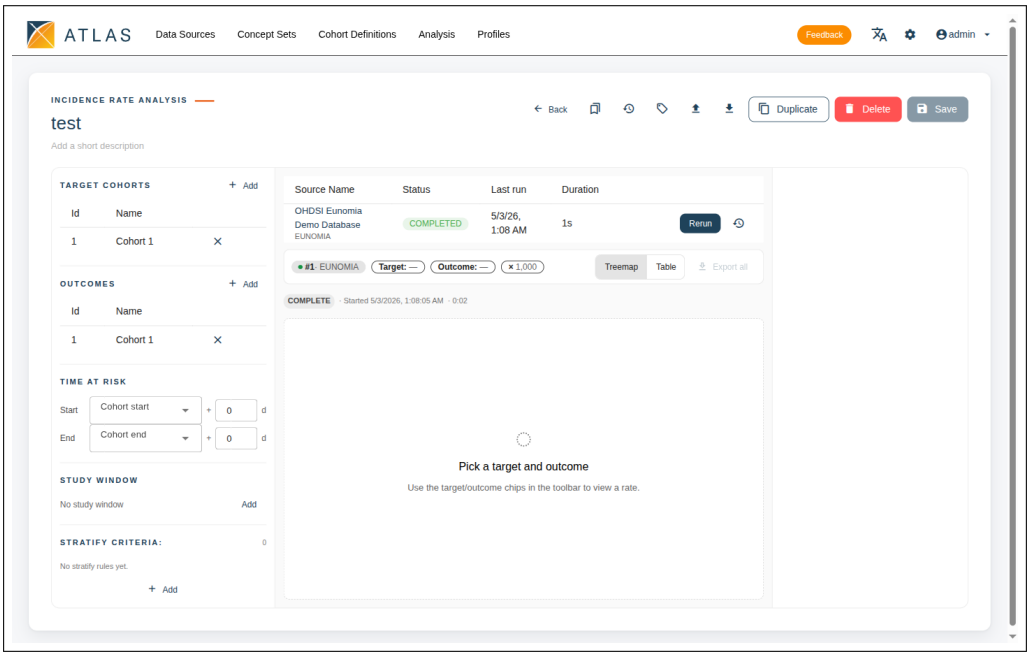


Figure 10.1. Incidence-rate editor. The design rail on the left holds Target cohorts, Outcomes, Time at risk (Cohort start / Cohort end + offset), Study window, and Stratify criteria. The canvas on the right shows the data-source run table above the result with the Target / Outcome / multiplier toolbar chips and the Treemap / Table toggle.

The time-at-risk window is the most consequential setting in this editor. Different choices (intent-to-treat vs per-protocol, etc.) can swing the rate by a factor of two or more on the same cohort. Report the TAR start and end with your rate and run a sensitivity check; the trade-offs are covered in *The Book of OHDSI* [20].

10.2 Reading the result

The result page has a stratified table and a treemap, with a **Treemap / Table** toggle on the canvas toolbar to switch between them. The rates table columns are:

- **Stratum** — **Summary** for the headline row, then one row per stratify rule.
- **Persons** — persons at risk in that stratum.
- **Cases** — outcome occurrences inside the time-at-risk window.
- **TAR (years)** — total time-at-risk in person-years.
- **Rate / multiplier** — the incidence rate, expressed per the multiplier picked from the toolbar (the menu offers $\times 100$, $\times 1{,}000$, $\times 10{,}000$ and $\times 100{,}000$).

A summary panel beside the table shows the headline incidence proportion in plain English; the table itself does not carry a confidence-interval column.

The treemap view shows nested rectangles whose area is proportional to the persons-at-risk in each stratum and whose fill follows the incidence-rate colour scale — the fastest

way to spot strata where the rate is unusually high or low without scanning the table row by row.

The canvas shows a data-source run table at the top with one row per connected source. Each row carries status, last-run timestamp, duration, and per-row **Run**, **Cancel** and **Show history** actions. **Show history** on a row opens the **Previous runs** dialog scoped to that source, where you can reopen any historical run — the same dialog used by characterizations and pathways.

A high rate computed from a tiny denominator looks impressive in isolation. Always look at *persons at risk* alongside the rate; suppress or flag rows where it is below a sensible cell-count threshold.

CHAPTER 11

Profiles

The **Profiles** area lets you look at one specific person's longitudinal record on one specific data source. It is useful for sanity-checking a cohort definition (does my cohort really catch the right people?) and for clinical review of individual cases when the data agreement permits.

11.1 Finding a person

1. Open **Profiles** from the top navigation, or arrive via a "view profile" action from another page.
2. Pick a source from the dropdown, or pass it in the URL as `/profiles/{sourceKey}`.
3. Type a person ID — the URL becomes `/profiles/{sourceKey}/{personId}` and the timeline loads.
4. Optionally append a cohort ID (`/profiles/{sourceKey}/{personId}/{cohortId}`) — the timeline will then highlight that cohort's membership window for the person.

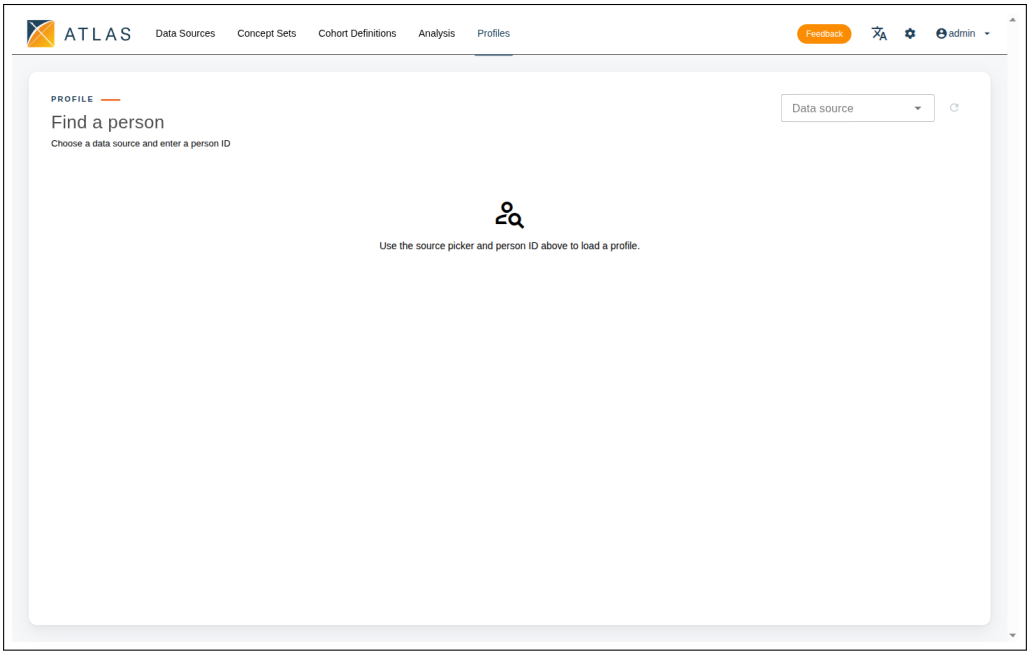


Figure 11.1. Profiles landing page. Pick a source from the dropdown top-right, then type a person ID (or arrive via a deep-linked URL) to load the timeline.

11.2 The timeline view

The main view is a horizontal time axis spanning the person's observation period, with one swim-lane per OMOP clinical domain (conditions, drug exposures, procedures, measurements, observations, visits). The x-axis is labelled *Day*: every record is plotted at its day-of-observation index relative to the start of the person's observation period, not at a calendar date.

Each lane is a sequence of marks at the day of each record in that domain; hovering a mark reveals the concept name and the day number. A row of filter chips above the chart lets you hide domains, and a brush on the timeline narrows the view to a day range; the active range appears as a closable chip ("Day $a \rightarrow$ Day b"). A separate events table below lists every record in detail with concept name, source code, and dates.

For phenotype validation, set the cohort context before opening the profile. The cohort window is shaded on the timeline so you can see at a glance whether the person's records around the index date match clinical expectations for the phenotype.

11.3 Validating a phenotype using Profiles

Profiles is the lightweight half of phenotype validation. The typical workflow is:

1. Generate the cohort against the source you want to validate on (Chapter 7).
2. Pick a small set of person IDs from the cohort — either by querying the cohort table directly, exporting the cohort result, or asking your administrator for a representative sample. Around a hundred persons is enough to find gross errors.
3. For each sampled person, open their profile with the cohort context set (`/profiles/{sourceKey}/{personId}/{cohortId}`). Read the records in the baseline window: do they look like patients with the condition? Read the records around the index date: is the entry event clinically plausible?
4. Tally what you see. Any obvious mismatches (a "type 2 diabetes" cohort that includes people with no glucose-lowering therapy and no diagnostic codes outside the index event) is a signal the rule is too loose; any common false negatives (clear diabetic patients excluded by an over-tight rule) is a signal it is too strict.
5. Adjust the cohort definition, save a new version (Chapter 13), regenerate, and resample.

Note

A sample of 100 charts is enough to find gross errors in a phenotype but not enough to estimate sensitivity, specificity, or positive predictive value with useful precision. For that, see the PheValuator section below.

11.4 Going further: PheValuator

ATLAS v3.0 itself does not formally evaluate a phenotype. For quantitative sensitivity, specificity, and PPV against a probabilistic gold standard, OHDSI ships *PheValuator* [18, 37, 38] as part of the HADES R suite [23]. The workflow is broadly: build the candidate phenotype and a tightly-specified "xSpec" cohort in ATLAS, export both as JSON, and run PheValuator in R against your CDM database — it fits a predictive model that yields per-source operating characteristics with confidence intervals.

The methodology, including how to construct the xSpec cohort, how to size the evaluation cohort, what target metrics make a phenotype publishable, and how PheValuator complements the `CohortDiagnostics` [28, 30] package, is covered in the package vignette and in *The Book of OHDSI*, chapter 16 "Clinical Validity" [22]. This manual documents what ATLAS provides; the evaluation step lives in R and outside the scope of the UI.

CHAPTER 12

Feature analyses

A *feature analysis* is a reusable definition of a covariate or summary statistic. Characterizations (Chapter 8) pull their content from feature analyses, and once a feature is defined once it can be reused across studies — which is what makes characterizations comparable to each other.

12.1 Built-in versus custom features

ATLAS v3.0 surfaces the feature analyses defined in WebAPI — including the standard library that corresponds to the OHDSI `FeatureExtraction` [15] R package (part of the HADES suite [23]): demographic features, condition occurrence in fixed windows, drug exposure in fixed windows, visit counts, Charlson index components, and so on. Most users start by combining these in characterizations rather than authoring new ones.

12.2 The three feature-analysis types

When you author a feature analysis from Analysis › Feature Analyses › New Feature Analysis, the editor asks for two orthogonal choices: a *type* that picks how the feature is expressed, and a *statistical type* that picks how its results are summarised.

The three types are:

Preset a JSON blob of FeatureExtraction `CovariateSettings` — the most common path for importing a community-standard feature set without re-deriving it. Useful when you want exactly what `createDefaultCovariateSettings(...)` would compute in R.

Criteria Set a small cohort-style criterion (concept sets, attribute filters, cardinality, temporal window). This is the type you reach for when the standard library does not cover a phenotype-specific feature you need to compute alongside the others.

Custom SQL a raw SQL template parameterised against the results schema. Reserved for features that cannot be expressed as a criteria set.

The two statistical types are picked separately:

Prevalence the result is a binary covariate per person (present / absent), shown as a percentage with a count column on the characterization result page.

Distribution the result is a continuous covariate, summarised as mean, standard deviation, median, and quartiles.

Note

A typical baseline bundle combines the two stat types: demographics (age as distribution, gender as prevalence), condition / drug / procedure occurrences in the four standard time windows (prevalence), visit and observation counts (distribution), and Charlson comorbidity index components. This default is what most published OHDSI studies use unless the question specifically calls for something narrower.

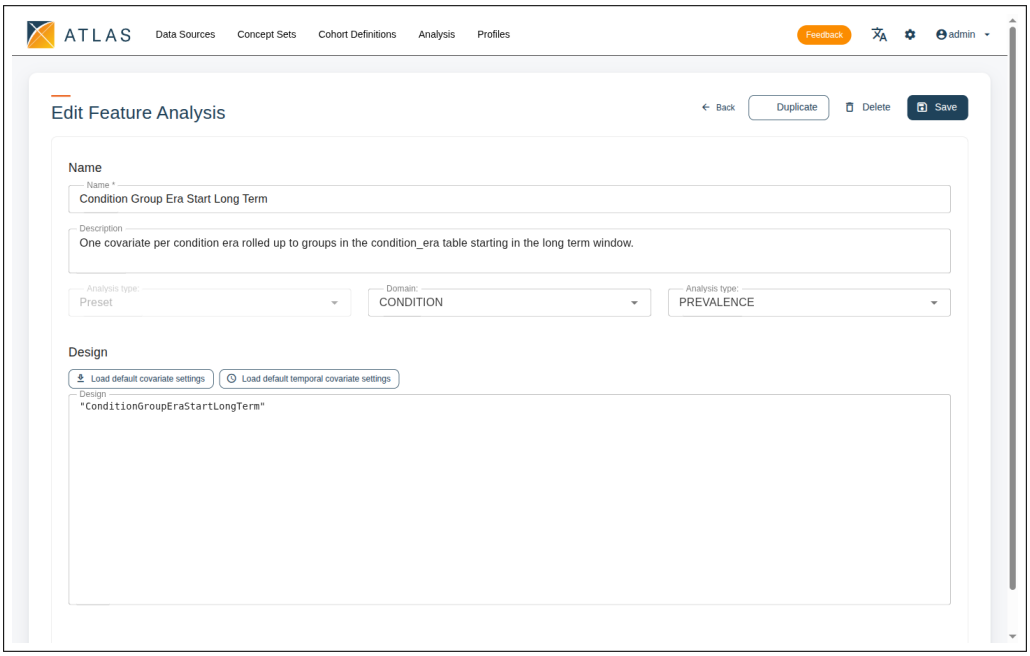


Figure 12.1. Feature Analysis editor. **Analysis type** (Preset / Criteria Set / Custom SQL) and **Stat type** (Prevalence / Distribution) sit alongside **Domain**; the Design block carries the JSON or SQL body appropriate to the chosen type.

12.3 Reuse

Once a feature analysis is saved, every characterization that references it picks up the latest definition automatically. Unlike cohorts and concept sets, feature analyses do not yet carry their own version history or per-feature tagging in the UI — the data model has those fields but the editor does not expose them. Treat each save as the canonical state, and use characterization-level versioning (Chapter 13) to pin a feature analysis as it appeared on a given run.

PART V

Operations

CHAPTER 13

Versions

Cohort definitions, concept sets, characterizations, pathways and incidence rates are all versioned. Every save creates a new version; older versions remain readable, and you can compare them, copy them, or restore them as the current working version. Feature analyses do not currently carry version history; their definitions are referenced directly by the characterizations that use them.

13.1 Where versions live

Each editable artefact carries a history (clock) icon in its toolbar that opens a **Versions** dialog. The dialog lists versions in reverse chronological order; each row shows the version number, author, save timestamp, and a short comment if one was supplied. A small badge on the icon shows the current version count at a glance.

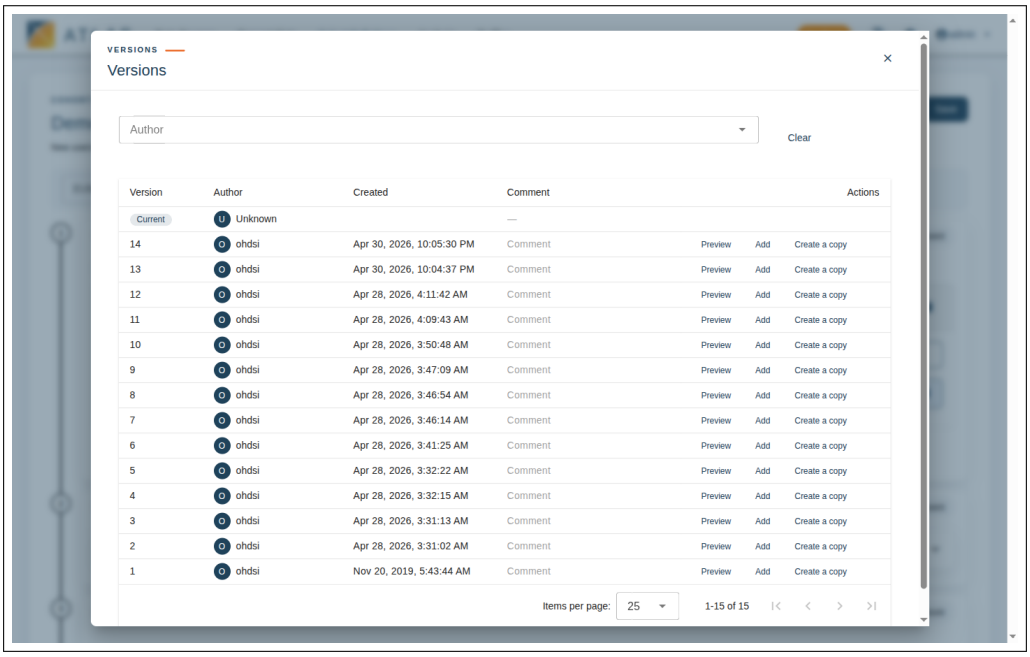


Figure 13.1. Versions dialog. Each historical version carries an author, timestamp, and optional comment, with **Preview**, **Add**, and **Create a copy** actions.

13.2 Previewing a version

Click a row in the Versions dialog to enter *preview mode*. The URL changes to e.g. `/cohortdefinition/{id}/version/{N}` and the editor renders the historical state.

Preview mode is read-only — you cannot accidentally overwrite the past — and a banner across the top of the editor makes it obvious you are not on the current version.

To leave preview mode, click **Back to current version** on the banner, or navigate to `.../version/current`, which returns you to the live working copy.

13.3 Restoring a version

From a preview, **Save as current version** writes the previewed state back as a new version on top of the current one. The historical row is preserved; you have not deleted history, you have appended a new entry that re-asserts the older content.

Note

Restoring is non-destructive on purpose. If you decide the restore was a mistake, the version immediately before it is still in the list.

13.4 Pinning a published study

When a cohort, concept set, or characterization is referenced in a published paper or a regulatory submission, pin the exact version in the citation: cohort definition ID and version number, not just "the latest version of cohort 482". This is the cheapest way to guarantee a reader on a different ATLAS instance — or on the same one a year later — can recreate the exact analytical artefact. The version-preview URLs (`/cohortdefinition/{id}/version/{N}`) are stable; copy them straight into the paper's supplementary materials.

Note

When the cohort itself was reused from the OHDSI Phenotype Library, pin *two* version numbers: the local cohort version and the Phenotype Library release DOI it was imported from. Definitions accepted into a Library release are immutable for that release, but the same definition can be marked deprecated [D] or in error [E] in a later release; quoting the release DOI insulates your study from those later changes [26].

CHAPTER 14

Administration

This chapter is for users with administrative permissions. Most researchers can skip it; come back when you need to grant a colleague write access, manage tags across the library, or check on long-running jobs.

14.1 The configuration panel

Administrative controls live in a side panel that opens from the **cog icon** in the top right of the navigation bar. The cog itself is gated: it is hidden entirely from users who hold none of the admin permissions, so a researcher account does not see it at all. Once open, the panel lists up to five tabs, each gated by a specific permission — a user with only one admin permission only sees one tab:

Cache (`admin:cache`). Patient and TrexSQL caches that per-source reports rely on. Trigger rebuilds, watch the progress, and review the timestamp of the last successful build. See "Caches and TrexSQL" below for what each cache does.

Data Sources (`admin:source`). The CDM databases WebAPI is connected to, listed with their source key, dialect, and connection status. This view is *read-only* — adding a source, changing a connection string, or rotating credentials happens against WebAPI's source tables directly (or via the Broadsea / atlasdb seed scripts).

Tags (`admin:tags`). Create, rename and merge the tags used across cohorts, concept sets and analyses.

Permissions (`admin:security`). The entry point for role and permission management. Selecting a role here opens the dedicated role page (covered below).

Jobs (`job:*:get`). The status view for long-running tasks — cohort generations, characterization runs, incidence-rate computations, pathway analyses, cache builds.

If a user with the cog visible somehow loses every admin permission, the panel renders a single info message — "You don't have access to any administrative settings." — in place of the tabs.

Note

The configuration panel is a side overlay rather than a separate page, so you can quickly check job status or cache state without leaving the page you were on.

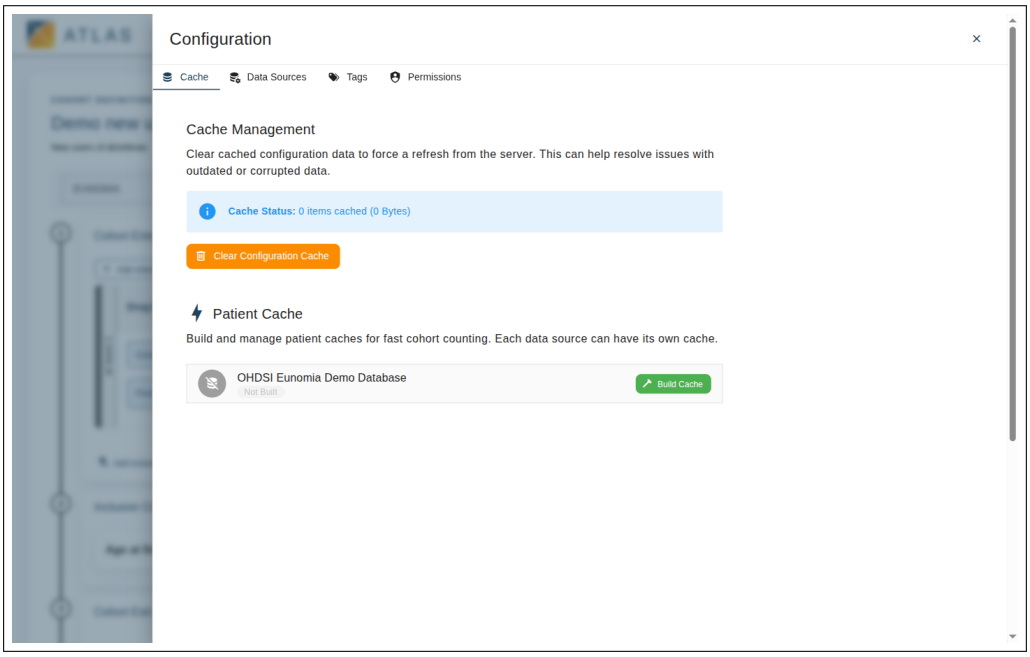


Figure 14.1. Configuration panel on the **Cache** tab. Cache management lives at the top; the Patient Cache section below has a per-source row with the cache state (here *Not Built*) and a **Build Cache** action.

14.2 Caches and TrexSQL

Several ATLAS pages back queries that are expensive to recompute on every visit — vocabulary searches, source-level descriptive reports, cohort report rollups. WebAPI handles those through two layers of cache that the configuration panel's **Cache management** tab exposes:

Patient cache. A per-source pre-aggregation that descriptive reports (Chapter 4, cohort **Reports** tab in Chapter 7) read from, and that the cohort builder uses to show *live patient counts* as a researcher edits a definition — saving a full **Generate** round-trip on every change. The cache sits in the source's results schema. Initial build is a long-running job; once built, it is reused until the source data is refreshed.

TrexSQL cache. A query-level cache used by WebAPI's SQL execution layer. Repeated vocabulary queries and concept-set lookups hit the cache rather than the database, so the second time a user opens the same concept set the response is essentially instant. The cache lives on disk on the WebAPI host (path set by the WebAPI environment variable `TREXSQL_CACHE_PATH`) and is shared across users.

Rebuild a cache after:

- the underlying CDM source has been refreshed (so the patient cache reflects the new data),
- the OHDSI vocabularies on the source have been updated (so the TrexSQL cache does not return stale concept information), or

- the cache was built against an older WebAPI version whose query shape has changed.

Cache rebuilds appear as jobs in the **Jobs** tab below; they can take from minutes to hours depending on the size of the source.

14.3 Roles and permissions

ATLAS v3.0 uses role-based access control. A *role* is a named bundle of *permissions*; a user is a member of zero or more roles. Open the configuration panel and pick **Permissions**, or navigate directly to `/config/roles` to see the role list.

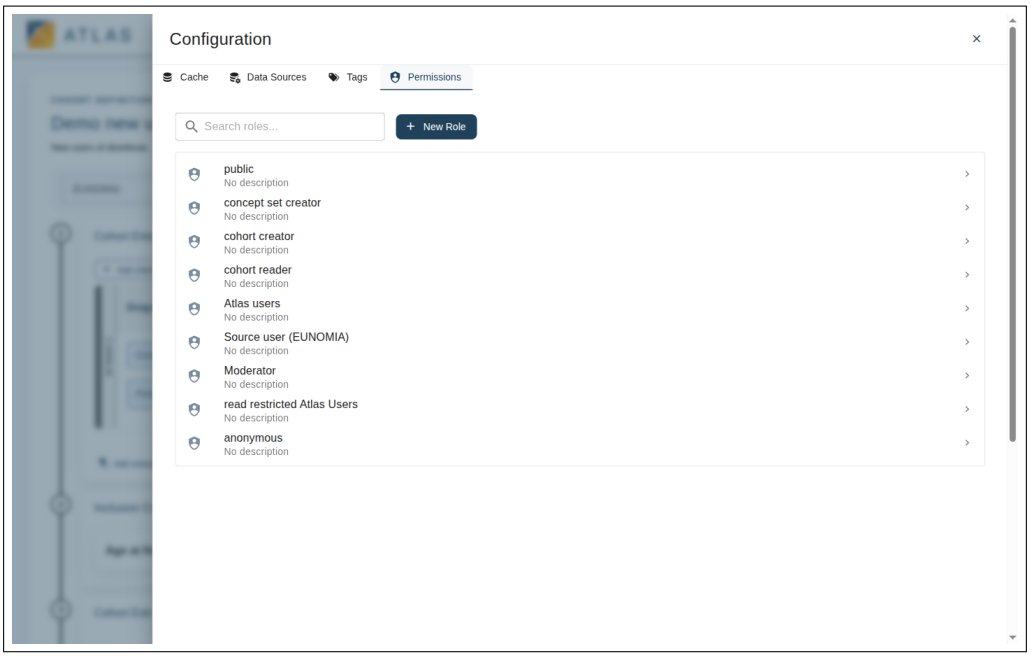


Figure 14.2. Roles list inside the **Permissions** tab of the configuration panel. Click any role to open the role-detail page (**Users** / **Permissions** / **Utilities** tabs); **New Role** starts a fresh empty role.

14.3.1 Built-in roles

A typical deployment ships a few built-in roles that you should not delete: `public` (default for unauthenticated visitors where allowed), `user` (read access to most artefacts), and an admin-style role with full write access. Your organisation may have added project-specific roles such as a per-study analyst role.

14.3.2 Editing a role

Click a role to open its detail page at `/config/roles/{id}`. The page lets you edit the role's name and description inline at the top, then exposes three tabs in this order (**Users** is the default):

Users The users currently in the role. Add or remove members here. Changes take effect at the user's next request, usually immediately.

Permissions The flags that decide what the role's members can do — view cohorts, write cohorts, generate against specific sources, manage roles, and so on. Most permissions are per-source, which is how you give a contractor access to one database without exposing the rest.

Utilities Import, export and clone operations on the role itself — useful when you want to copy a vetted role bundle to a sibling deployment.

14.3.3 Creating, importing and deleting a role

The role list carries three companion dialogs:

New role opens a small dialog with a name and description; the new role starts empty and you flesh it out from the detail page above. Pick a clear, hyphenated name (e.g. `cardiology-analyst`). Role names are visible to all users, so avoid embarrassing or project-internal language.

Import reads a role JSON file produced by another ATLAS instance's **Utilities** tab. It is the right way to replicate a vetted role bundle from one deployment to another.

Delete removes the role after a confirmation dialog. Built-in roles are typically protected; only delete a role you own.

Removing a permission from a role takes effect immediately for every member of that role. If a long-running cohort generation is in flight and the role member loses generation permission mid-flight, the run will fail. Coordinate sensitive role changes with your users.

14.3.4 Permission reference

The permission strings ATLAS checks fall into four families. The table below lists the ones the frontend reads when deciding what to render.

Resource verbs `read:⟨resource⟩`, `write:⟨resource⟩`, `create:⟨resource⟩`, where *resource* is one of `cohort-definition`, `conceptset`, `cohort-characterization`, `feature-analysis`, `pathway`, `incidence`, or `reusable`. Read gates the list and editor; write gates **Save** and **Delete**; create gates the **New** ... button on the list page.

Per-entity grants alongside the global verbs, WebAPI returns a per-entity grant map (`READ`, `WRITE`, `OWNER`) for each artefact the user has been granted access to individually. Ownership implies write; an explicit `WRITE` grant overrides a missing global `write:` permission. This is how you give a researcher full control over their own cohorts without granting blanket `write:cohort-definition`.

Admin `admin:cache` (Cache tab and patient-cache rebuilds), `admin:source` (Data Sources tab), `admin:tags` (Tags tab and tag-management actions on entities),

`admin:security` (Permissions tab, role CRUD; also implies read+write on every entity). Holding any of these makes the cog icon visible in the navbar.

Source access `source:*:access` gates which CDM sources a user can run cohort generations and reports against. Per-source variants (`source:⟨key⟩:access`) can grant access to a single source without the wildcard.

Jobs `job:*:get` gates the Jobs tab in the configuration panel.

Note

A common starter-role bundle for an analyst is `read:*`, `write:*` on the artefact resources plus `create:*`, `source:*:access`, and `job:*:get` — everything except `admin:*`. A read-only viewer role drops the `write:*` and `create:*` entries. Per-entity grants then layer on top to share specific cohorts with collaborators outside the analyst pool.

14.4 Jobs and long-running tasks

The **Jobs** section in the configuration panel is the status view for every long-running task in the system: cohort generations, characterization runs, incidence-rate computations, pathway analyses, and the per-source caches. Each row shows the job name, target source, status (**RUNNING**, **COMPLETED**, **FAILED**), submitted-by user, start and end times, and duration.

Refresh re-fetches the list from WebAPI — useful when you are watching a long generation finish.

Filtering by status lets you focus on running jobs or recently failed ones.

Clicking a row opens the job's log, where errors and SQL warnings are reported verbatim.

The Jobs section is the right place to diagnose "why is my generation taking so long" and "did my characterization actually finish". For administrators, scanning the failed-jobs filter periodically catches WebAPI-side problems (connection timeouts, out-of-memory errors) before users notice them.

14.5 Audit information

ATLAS v3.0 stores the author and last modifier on every artefact and shows both in the list views. Together with the version history (Chapter 13) this is the basic audit trail. Deeper audit logs — of generation runs and login events — live in the WebAPI database and are typically inspected directly there rather than through the UI.

CHAPTER 15

Plugins

ATLAS v3.0 supports plugins — independently developed front-end modules that appear inside the application as additional navigation entries and pages. From a user's perspective a plugin behaves like any other area of ATLAS v3.0; from an administrator's perspective it is a separately released JavaScript bundle that the host loads at runtime.

15.1 What plugins look like

When your deployment has plugins installed, they appear in the top navigation alongside the built-in entries. The content area below the nav is replaced with the plugin's own UI when you select it. Plugins share the active session, so you do not log in again, and they receive the currently selected data source automatically.

Figure 15.1. Top navigation on a deployment without plugins; loaded plugins would appear as additional entries to the right of **Profiles** (or replace built-in entries when configured to).

15.2 Common plugin categories

A given deployment may ship none, some, or all of these:

- **Custom analytics.** Statistical workflows specific to a research network, exposed as a plugin so they live alongside ATLAS v3.0's built-in analyses without forking the application.
- **Connectors.** Bridges to external systems — a redcap export, a SAS dataset writer, a registry submission tool.
- **Visualisations.** Dashboards or chart libraries that consume cohort or characterization output.

15.3 Trust and isolation

Each plugin runs in its own sandbox with a scoped message bus to the host. Plugins receive only the auth context and the API surface the host explicitly exposes; they cannot reach into stores they have not been granted. From a research integrity standpoint this matters, because a plugin that produced study results becomes part of the chain of custody — knowing exactly what version of which plugin produced a chart is something your audit process should care about.

Note

There is no in-application admin UI for installed plugins; the plugin manifest is a static `src/config/plugins.json` on the host, set at build or deploy time. If you are reviewing a publication that used ATLAS v3.0, asking the deployment operator for that file — and the build versions of the plugins it references — is a fair request.

15.4 The `plugins.json` manifest

The host loads its plugin manifest from `/config/plugins.json` at startup; the file is validated with a Zod schema (`PluginManifestSchema` in `src/models/PluginModels.ts`). A minimal manifest looks like this:

```
{
  "version": "1.0",
  "plugins": [
    {
      "id": "hello-world-plugin",
      "name": "Hello World",
      "version": "1.0.0",
      "entryPoint": "hello-world-plugin/index.js",
      "menuItems": [
        {
          "id": "main",
          "name": "Hello World",
          "route": "/plugins/hello-world-plugin/main",
          "icon": "mdi-hand-wave",
          "order": 0
        }
      ],
      "metadata": {
        "author": "Atlas Team",
        "description": "Demo plugin",
        "homepage": "https://github.com/ohdsi/atlas"
      }
    }
  ]
}
```

The relevant fields:

`version` manifest format version (currently `"1.0"`).

`plugins[]` one entry per plugin. Each plugin specifies an `id` (unique within the manifest), a human-readable `name`, the plugin's `version`, and an `entryPoint` relative to the deployment's *plugins path* (default `plugins/`). The plugin's

`menuItems[].route` **must** start with `/plugins/⟨id⟩/` — the host rejects the manifest otherwise.

`metadata` optional descriptive fields; surfaced nowhere user-facing but useful for audit queries.

The manifest also has an optional `settings` block that tunes the host without touching code:

`enableHotReload` reload plugins on bundle changes (development only; default `true`).

`loadTimeout` milliseconds to wait for a plugin bundle before failing the load (default 30000).

`pluginsPath` base path the `entryPoints` are resolved against.

`showLoadingIndicators` whether the host shows a spinner while a plugin's bundle is fetching.

`navigation.enabledCoreItems` / `disabledCoreItems`: explicit lists used to hide built-in nav entries on a deployment that does not need them. `disabledCoreItems` takes precedence.

`theme.primaryColor`, `theme.logoUrl`, `theme.logoNavigateTo` colour and logo overrides.

`header.show`... toggles for the navbar slots: `showNavBar`, `showFeedbackButton`, `showLanguageSelector`, `showConfigButton`, `showUserMenu`; plus `header.feedbackUrl` for the external feedback link target.

A common deployment hygiene check is "does the manifest only list plugins we expect to ship?" — since the manifest is checked into deployment config, a code review on `plugins.json` catches the same supply-chain risks as a code review on the application itself.

15.5 Errors and troubleshooting

If a plugin fails to load, the host shows a placeholder card in place of the plugin's UI with the failure reason. The most common causes are network errors fetching the bundle, version skew between plugin and host, and configuration errors in `plugins.json`. Reload the page after the operator has redeployed; if the problem persists, open the browser developer console and forward the single-spa lifecycle errors to your administrator.

CHAPTER 16

Troubleshooting

The most common problems users hit on ATLAS v3.0, and how to recognise them. Items are grouped by symptom rather than by chapter; if you know what is wrong already, jump to the relevant reference chapter for the long form.

16.1 I cannot log in

Symptom: the credentials I usually use are rejected. Check the WebAPI clock. ATLAS uses JWT tokens signed by WebAPI; if the WebAPI host's clock has drifted more than a minute or two relative to the browser, every freshly issued token looks already-expired and the login appears to fail (Chapter 2).

Symptom: my OAuth/SAML/OIDC button is missing. The login providers are configured on the WebAPI side. Ask your administrator whether `SECURITY_AUTH_*` for that provider is set to `true`; on a fresh deployment only DB-auth is enabled by default (Appendix A).

Symptom: I am repeatedly logged out mid-task. The token lifetime is shorter than your session. ATLAS shows an extend-session dialog before the token expires; if the page is not active the token quietly expires. Auto-saved cohort drafts are restored from session storage on next login, but unsaved analysis runs are lost.

16.2 I cannot see any data

Symptom: the Data Sources page lists my source but every report tab is empty. The OHDSI Achilles routine has not been run against that source's results schema. Atlas reads from the Achilles result tables; if they are missing, every tab renders blank even when the source itself is connected. This is the single most common "set up but nothing shows" cause. See Appendix A for how to run Achilles.

Symptom: the Data Sources page is empty. WebAPI has no sources registered. Check `<host>/WebAPI/source/sources` — if it returns an empty JSON array, no source rows exist. Add them via the WebAPI database tables or the `atlasdb` seed scripts (Appendix A).

Symptom: per-source record-count columns or live patient counts stay blank. Those numbers come from the per-source TrexSQL patient cache. If the cache has

not been built (or is stale after a CDM refresh), they remain dashes. An administrator can rebuild the cache from the configuration panel's **Cache** tab; cache builds run as long-running jobs visible in the **Jobs** tab (Chapter 14).

16.3 Cohort generation does not produce what I expect

Symptom: the cohort generates zero rows. Open the **Inclusion Rules** tab inside the Generation panel. The first row is the entry-event count; if it is already zero, your concept set has no records on this source. Either the concept set resolves to nothing (open the set's **Included Concepts** tab and check the per-row record-count columns), or the entry-event window or attribute filters exclude every candidate. Run the cohort against a source you know has the relevant data first.

Symptom: the cohort produces vastly more rows than expected. Almost always one of: (i) **Cohort entry on** is set to **all** when you wanted **earliest event**, multiplying the row count; (ii) a concept set has **Descendants** ticked on a parent that pulls in unintended children (Chapter 6).

Symptom: an inclusion rule drops nobody. The rule is either trivially satisfied (cardinality too lax, temporal window too wide) or redundant with an earlier rule. Tighten or remove. The inclusion-rule debugger guidance in Chapter 7 walks through this.

Symptom: the same JSON produces wildly different counts on two sources. Almost always an ETL or vocabulary-mapping difference, not a real clinical signal. Run the same cohort on a third source (Chapter 7); if its attrition pattern matches one of the two but not the other, the outlier is the ETL-suspect source.

Symptom: generation takes much longer than expected. Open the configuration panel's Jobs tab (cog icon → Jobs) and inspect the running job's log. WebAPI prints SQL warnings and database-side errors verbatim. Common causes: a concept set with thousands of resolved members hammering CONDITION_OCCURRENCE, or a missing index on the results schema (an administrator concern).

16.4 Concept set issues

Symptom: the search returns no results for a term I know is in the data. Check the Vocabulary filter — if it excludes the vocabulary your data uses you will see nothing. ICD codes live in the ICD vocabulary; SNOMED standards live in SNOMED; non-US drugs live in RxNorm Extension rather than RxNorm. Clear the filter and re-search if uncertain.

Symptom: a concept set with Descendants ticked covers thousands of unrelated terms. Either the parent concept is too high in the hierarchy — **Descendants** pulls down everything — or the parent concept is itself non-specific. Move down the hierarchy

or narrow the parent. The descendant record count (**DRC**) on the row in the **Included Concepts** tab is your in-place sanity check.

Symptom: my concept set looks right but the cohort matches nobody. The source's `*_concept_id` columns may be populated from a different vocabulary than the one you selected (e.g. the ETL maps to SNOMED while you searched on ICD). Toggle **Mapped** on the rows in the **Included Concepts** tab so source codes that map up to your standard concept also count, and confirm the record-count columns on the row become non-zero before re-generating.

16.5 Permissions: I cannot see or do something

Symptom: the cog icon for the configuration panel is not visible. The cog is hidden from users who hold none of the admin permissions (`admin:cache`, `admin:source`, `admin:tags`, `admin:security`, `job:*:get`). Ask your administrator to add you to a role that carries the relevant permission (Chapter 14).

Symptom: the configuration panel opens but only some tabs are present. Each admin tab is gated by its own permission, so a role with only `admin:cache` sees only the **Cache** tab. The other tabs are not disabled; they are not rendered at all. This is expected.

Symptom: Save or Delete is disabled on a cohort or concept set I am editing. You do not hold write permission on this entity. Either you are not the author and your role lacks the relevant `cohort:*:put` / `conceptset:*:put` permission, or the entity is locked by another concurrent edit. Confirm with your administrator.

16.6 Errors that show up in the browser console

CORS errors. The frontend is on a different origin from WebAPI, and WebAPI's `SECURITY_CORS_ALLOWED_ORIGINS` does not list the ATLAS origin. Add the origin or put both behind the same reverse proxy (Appendix A).

401 Unauthorized. The bearer token has expired or was rejected. Reload the page; ATLAS will prompt for a fresh login.

404 Not Found on a /WebAPI route. The frontend's `VITE_WEBAPI_URL` (or the reverse proxy's `/WebAPI` forwarder) points at the wrong host. The appendix's verify-step is to call `<host>/WebAPI/info` and check the JSON response.

500 Internal Server Error on a /WebAPI route. A WebAPI-side problem. The response body usually carries a human-readable error; if not, check the WebAPI log.

16.7 Plugin problems

Symptom: a plugin entry in the navigation shows an error card instead of its UI. The plugin bundle failed to load. Common causes: a network error fetching it, version skew between plugin and host, or a configuration error in `plugins.json` (Chapter 15). The browser developer console usually carries a single-spa lifecycle error that points at the issue.

16.8 When the answer is not in this manual

- Live WebAPI reference: `<host>/WebAPI/swagger-ui/` (covered in Appendix A).
- Atlas3 source / issue tracker: github.com/OHDSI/Atlas3.
- OHDSI community forum: forums.ohdsi.org.
- The Book of OHDSI [20] for the methodology behind any feature.

APPENDIX A

Setup guide: WebAPI and ATLAS v3.0

This appendix walks through a working installation of WebAPI and the ATLAS v3.0 frontend. The goal is to get a developer or evaluator from zero to a running web application that talks to a real PostgreSQL backing store. For production deployments, follow your organisation's infrastructure standards rather than the recipes here — this chapter is intentionally minimal.

ATLAS v3.0 requires **WebAPI v3.0**. The 2.x line of WebAPI that the production ATLAS uses — and that ships with the OHDSI Broadsea distribution — is *not* compatible. The v3.0 backend introduces schema and endpoint changes the frontend relies on, and the frontend assumes the v3.0 contract from its first request. When you clone WebAPI, check out the v3.0 branch (e.g. `3.0-dev`) explicitly; the development docker compose in the Atlas3 repository pulls `ghcr.io/ohdsi/webapi:3.0-dev` for exactly this reason.

A.1 The pieces

A working ATLAS v3.0 deployment has three independent components:

A relational database holding the OHDSI WebAPI schema, the OMOP Common Data Model schemas you want to analyse, and any result schemas. PostgreSQL is the simplest choice and the one the development `docker-compose.yml` ships with; Microsoft SQL Server, Oracle, Amazon Redshift, and Snowflake are all supported by WebAPI.

The WebAPI service a Java/Spring application that exposes the REST endpoints ATLAS talks to, executes cohort SQL against the configured CDM sources, and runs Flyway migrations against its own schema on first boot.

The ATLAS v3.0 frontend a Vue 3 single-page application, served either by the Vite development server or as a static bundle behind a reverse proxy.

In a Docker Compose setup these are three services. In a production deployment they may be one database cluster, one Java application server, and a static-asset host.

A.2 Prerequisites

- Docker and Docker Compose v2 (the fastest path).
- *or* for native installs: PostgreSQL 12+, JDK 17, Maven 3.9+, Node.js 18+ and `npm`.
- Git.
- Roughly 4 GB of free disk for container images and the WebAPI Flyway baseline.
- A WebAPI build from the v3.0 line — container image `ghcr.io/ohdsi/webapi:3.0-dev` or a local build of the matching branch from the WebAPI repository.

A.3 The Docker Compose path (recommended for evaluation)

The ATLAS v3.0 repository ships a `docker-compose.yml` that brings up Postgres, WebAPI, a database initialiser, and the frontend behind Caddy with a self-signed certificate. From a fresh checkout:

```
git clone https://github.com/OHDSI/Atlas3.git
git clone --branch 3.0-dev https://github.com/OHDSI/WebAPI.git
cd Atlas3
docker compose up
```

The compose file expects `WebAPI` to live in a sibling directory because it builds the WebAPI image from `../WebAPI`; cloning the `3.0-dev` branch is essential because ATLAS v3.0 will not work against the WebAPI 2.x line. The first boot takes a few minutes while WebAPI runs its Flyway migrations into the `webapi` schema and the `atlasdb` initialisation scripts seed a default user.

When the stack is healthy:

- ATLAS v3.0 is at `https://localhost` (accept the self-signed certificate).
- WebAPI is at `http://localhost:8080/WebAPI/info`; that endpoint returns a JSON blob with the version once the service is up.
- PostgreSQL is published on host port 5433 for external clients (the in-network port is the standard 5432).

The default credentials match the values seeded by the `atlasdb` scripts; check that folder for the actual username and password and *change them before any non-local use*.

The default `POSTGRES_PASSWORD` in the compose file is `mypass`. It is a placeholder for local development. Set `POSTGRES_PASSWORD` in a `.env` file (or your secrets manager) before running anywhere a hostile network can reach.

A.4 Native install

If you want each component on the host without Docker, the steps below mirror the compose recipe.

A.4.1 1. Database

1. Install PostgreSQL 12 or newer.
2. Create a database (`ohdsi` is a common choice) and a `webapi` schema.
3. Create a role with `CREATE` privileges on that schema — WebAPI needs to run Flyway migrations against it on startup.
4. Optionally, restore one or more OMOP CDM databases as additional schemas. These are the data sources the application will offer in **Data Sources**.

A.4.2 2. WebAPI v3.0

1. `git clone --branch 3.0-dev https://github.com/OHDSI/WebAPI.git`. The default branch of the WebAPI repository tracks the production 2.x line, which ATLAS v3.0 cannot talk to; explicitly check out the v3.0 branch.
2. Create a Maven settings profile that maps the database connection details to the property keys WebAPI expects. The compose file lists the relevant variables verbatim — see `atlas3-webapi.environment` in `docker-compose.yml` for the canonical names (`DATASOURCE_URL`, `DATASOURCE_USERNAME`, `DATASOURCE_PASSWORD`, `DATASOURCE_OHDSI_SCHEMA`, the `SPRING_FLYWAY_*` keys, and the `SECURITY_AUTH_DB_DATASOURCE_*` keys for database-backed authentication).
3. Build with `mvn -P webapi-postgresql package`; the result is `WebAPI.war` in `target/`.
4. Deploy the WAR into Tomcat 9 or run the embedded Spring Boot launcher. WebAPI will run Flyway migrations against the configured schema on first boot.
5. Confirm `http://localhost:8080/WebAPI/info` returns the version JSON.

A.4.3 3. Add data sources to WebAPI

WebAPI reads its source list from a database table populated by the `atlasdb` initialisation scripts (or by the Broadsea stack). Each row points WebAPI at one CDM database with a JDBC URL, schema names for CDM and results, and a friendly source key. Until at least one source exists, the ATLAS v3.0 **Data Sources** page is empty.

A.4.4 4. ATLAS v3.0 frontend

1. `git clone https://github.com/OHDSI/Atlas3.git && cd Atlas3`.
2. `npm install`.
3. Configure the WebAPI URL and authentication providers. The relevant Vite environment variables are `VITE_WEBAPI_URL`, `VITE_AUTH_ENABLED`,

`VITE_AUTH_SKIP_LOGIN`, and `VITE_AUTH_PROVIDERS`. Put them in a `.env.local` file or pass them at build time.

4. For development, run `npm run dev`; the dev server proxies `/WebAPI` requests to whatever URL `VITE_WEBAPI_URL` resolves to. Default proxy target is the public OHDSI demo at https://atlas-demo.ohdsi.org/WebAPI.
5. For production, run `npm run build` and serve the contents of `dist/` from a static host. A reverse proxy must forward `/WebAPI` to the Java service so the same origin serves both. The repository's `Caddyfile` shows a working configuration.

Note

`npm run dev` is the right entry point for trying things out or for plugin development; it runs the Vite dev server with hot module reload. The compose stack instead builds the static bundle into the Caddy image, which is what a production deployment would do.

A.5 First-time login and creating a user

The default DB-auth path expects a row in the `webapi.auth_user` table whose password column is a bcrypt hash of the chosen plaintext. The compose seed scripts in the `atlasdb/` folder create an initial administrator. To add more users, either insert rows into `webapi.auth_user` directly or, if your deployment exposes LDAP/OAuth/SAML/OIDC, configure the corresponding `SECURITY_AUTH_*` group of properties on the WebAPI side and let users log in via that flow instead.

After signing in, an administrator should:

- visit Configuration → Permissions and set up the role bundles their organisation needs (Chapter 14);
- confirm the **Data Sources** page lists each connected CDM and that its dashboard renders without errors (Chapter 4);
- run a small smoke-test cohort against each source to validate end-to-end execution (Chapter 7).

A.6 Common installation issues

The ATLAS page loads but every list is empty. WebAPI is reachable but no CDM sources are configured. Add rows to the WebAPI source tables (or run the `atlasdb` init scripts).

CORS errors in the browser console. The frontend and WebAPI are on different origins and WebAPI's `SECURITY_CORS_ALLOWED_ORIGINS` does not include the ATLAS v3.0 origin. Either add the origin to the allow-list or put both behind the same reverse proxy.

Flyway error on WebAPI startup. The configured database user lacks privileges on the `webapi` schema, or a previous WebAPI version left a migration in an inconsistent state. Inspect the WebAPI log; Flyway prints the failing migration explicitly.

Login fails with valid credentials. Check the WebAPI clock against the host running the browser. A skew larger than the JWT clock-tolerance window makes every freshly issued token look already-expired.

Data Sources page is empty for every tab. The most common case: the source is configured but *Achilles* [9] has not been run against it. ATLAS v3.0's descriptive reports read from the Achilles results tables in the source's results schema; if those tables are missing, every tab renders empty even though the source itself is reachable. See the next section.

A.7 Verifying a clean setup

After the stack is up, run these three checks before declaring victory:

1. `curl -sk https://localhost/WebAPI/info` should return JSON containing the WebAPI version and build date.
2. `curl -sk https://localhost/WebAPI/source/sources` should return a non-empty JSON array with one entry per configured CDM. An empty array means no sources are registered, and every ATLAS v3.0 list page that depends on a source will appear empty.
3. In the browser, open the developer console (Ctrl+Shift+I in Chromium) and watch the Network tab while loading **Data Sources**. You should see 200 responses for the `/source/sources` call and for each per-source report. Repeated 401 responses point at an authentication problem, 404 at routing, 500 at WebAPI itself — the response body usually has a clear error.

A.8 Achilles: the missing prerequisite for descriptive reports

OHDSI Achilles [9] is a separate R package that analyses a CDM database and writes summary tables into the same source's results schema. ATLAS v3.0's **Data Sources** reports *read from those Achilles tables*; if Achilles has never run, the source is connected but the dashboards are blank. Most evaluators hit this once and remember it forever.

For each source you want to use:

1. Install Achilles in an R environment that can reach the CDM: `install.packages("Achilles")` (or via `remotes::install_github("OHDSI/Achilles")`).
2. Run `Achilles::achilles(connectionDetails, ...)` with the CDM and results schema names. On a multi-million person database this can take from minutes to hours.
3. Re-run Achilles after every CDM refresh; the results schema is rebuilt from scratch each time.

Two adjacent OHDSI tools you may want to install at the same time: the Data Quality Dashboard [14] for systematic data-quality checks against the CDM (Chapter 4), and PheValuator [18, 37] for formal phenotype evaluation (Chapter 11).

A.9 Related: Broadsea

If the recipe in this appendix feels too low-level, the OHDSI Broadsea [13] project bundles WebAPI, ATLAS, Achilles, the Data Quality Dashboard, and a sample CDM into a single Docker stack with a guided setup. It is the fastest way to demo the whole OHDSI toolchain on a laptop.

The current Broadsea release ships ATLAS 2.x and the matching WebAPI 2.x backend, neither of which works with ATLAS v3.0. Broadsea support for ATLAS v3.0 / WebAPI v3.0 is on the OHDSI roadmap and expected in an upcoming release; until that lands, swapping in the v3.0 components manually is required and non-trivial (the database seed scripts and the WebAPI image both need to change in lockstep). For a v3.0 stack today, use the `docker-compose.yml` that ships in the Atlas3 repository — it is the supported path and what the rest of this appendix documents.

A.10 Further reading and API reference

This manual documents what users do in the ATLAS v3.0 interface; it is not a reference for the WebAPI backend that the UI talks to. For power users who want to call WebAPI directly — to script cohort generation, batch concept-set imports, or build a plugin against the same endpoints the UI uses — the canonical references live upstream:

ATLAS v3.0 frontend repository github.com/OHDSI/Atlas3 — source, issue tracker, build instructions.

WebAPI repository github.com/OHDSI/WebAPI — the v3.0 backend; check out the `3.0-dev` branch for the version this manual documents.

Live WebAPI reference `<host>/WebAPI/swagger-ui/` — the deployed instance's own swagger UI lists every endpoint with request and response shapes. On a local Docker Compose stack the URL is `http://localhost:8080/WebAPI/swagger-ui/`.

Book of OHDSI ohdsi.github.io/TheBookOfOhdsi — the conceptual reference for OMOP / ATLAS / the OHDSI toolchain. Cited throughout this manual.

The bibliography section at the back of the manual lists the papers and packages cited from individual chapters, including HADES [23], FeatureExtraction [15], Achilles [9], and PheValuator [37].

APPENDIX B

For users coming from ATLAS 2.15

ATLAS v3.0 is a complete reimplementation of OHDSI ATLAS (the 2.x line, current production version 2.15) on Vue 3 and TypeScript. The underlying WebAPI, the cohort JSON, the concept sets and the analyses are the same; the user interface is different. This appendix is a quick translation table for users who are fluent in 2.15 and want to find the equivalent control in v3.0 without re-reading every chapter.

B.1 Reorganised navigation

Concept Sets (top nav) is now a *tab* on a **Concepts** page that also hosts **Concept Search**.

Profiles is a first-class top-nav entry, no longer only reachable via deep link from another page.

B.2 Renamed buttons and toggles

The user-facing labels are loaded from WebAPI's translation bundle, so on a typical deployment most 2.x labels are unchanged. The renames below are the ones that did move.

New inclusion criteria → Add rule.

Combination window (pathways) → Collapse window.

All Executions (every analysis) → per-source Show history action on the data-source run table, whi

Configuration → Roles → Configuration → Permissions.

B.3 Removed concepts and tabs

Cohort Reports tab. No longer exists in the cohort builder. The cohort-specific reports that lived there are now reached from the **Generation** side panel (**Inclusion Rules** and **Samples** tabs); the prevalence-style descriptive reports live on the **Data Sources** page (Ch 4).

Cohort Generation tab. Replaced by a **Generate** button on the toolbar that opens a side panel.

Concept-set Concept Set Expression / Included Source Codes tabs. Removed. The editor now has **Included Concepts** (the resolved per-row view, with descendants/mapped/exclude flags) plus sibling **Search**, **Recommend**, and **Compare** tabs. The Recommend (PHOEBE) and Compare (Venn) tabs are new — see Ch 6. The Included Source Codes view is folded into the per-row record-count columns on the Included Concepts tab.

Concept-set Export / Import buttons. Replaced by export through the editor's standard JSON download flow at the cohort level; concept-set JSON is embedded in the cohort's exported JSON.

Cohort Export to SQL. Removed; use the WebAPI endpoint `/cohortdefinition/⟨id⟩/sql` if you need the compiled SQL.

Versions *tab*. Versions now open as a *dialog* from a clock icon on the editor toolbar (Ch 13).

B.4 New features that have no 2.15 equivalent

Live patient-count bar. Pinned to the top of the cohort builder; shows a running cohort estimate as you edit. Requires TrexSQL and a built patient cache. See Section 7.1.

Recommend tab in concept-set editor. PHOEBE 2.0 recommendations for additional concepts based on co-occurrence patterns across an OMOP network (Ch 6).

Compare tab in concept-set editor. Side-by-side Venn-and-table comparison of two saved concept sets (Ch 6).

Paste IDs. Bulk-add concepts to a concept set by pasting an ID list (Ch 6).

Cohort Samples tab. In the Generation panel; draw a random subset of the generated cohort with optional age and gender filters, click through to profiles. See Section 7.8.1.

Tile / Table view toggle on the Cohorts list. (The Cohorts list page only.)

Permission-gated configuration. The cog icon is hidden when the user has no admin permission, and each configuration tab is gated by a separate `admin:*` permission. See Section 14.3.4.

Plugin system. Single-spa-based dynamic plugins configured through `plugins.json` on the host (Ch 15).

B.5 Things that look different but behave the same

Cohort builder layout. Numbered step rail, but the underlying logic (entry event → inclusion → exit → censoring) is unchanged. The cohort JSON is identical to 2.15's.

Vocabulary search columns. The two record-count columns have been split into four: **RC** / **DRC** for record counts, **PC** / **DPC** for person counts. Same data; finer-grained presentation.

Characterization editor. Same target cohorts plus feature analyses model; what 2.15 called "strata" appears under the label **Subgroup analyses** in v3.0, and there is no separate "outcome cohorts" slot — outcome-stratified characterisations are expressed as target + matching subgroup.

B.6 Migration notes

Cohort JSON, concept-set JSON, characterization JSON, pathway JSON and incidence-rate JSON exported from ATLAS 2.x are accepted by v3.0 (a converter normalises them on import). Round trips are lossless for the fields v3.0 supports; a small list of 2.x-only fields is dropped on import. Run a known-good cohort through the import path and **Generate** on the same source as the 2.x reference run to verify byte-equal counts before relying on a converted definition for a study.

Glossary

CDM (Common Data Model) The OMOP Common Data Model is a person-centric relational schema that standardises observational health data so the same analytic code runs against any compliant dataset.

Cohort A set of persons who satisfy one or more inclusion criteria for a duration of time. In ATLAS v3.0 a cohort is defined by an entry event, optional inclusion rules, and an exit rule.

Concept A single entry in the OMOP standardised vocabularies, such as a SNOMED condition, an RxNorm ingredient, or a LOINC measurement.

Concept set A reusable, named collection of concepts, with options to include descendants, include mapped source codes, and exclude specific concepts.

Standard concept A concept the OHDSI vocabulary marks as canonical for its domain. Source codes (ICD-10, NDC, etc.) usually map up to a standard concept. Cohort logic should reference standard concepts so it is portable across data sources.

Era A continuous period of clinical state derived from underlying events — for example, a drug era is built from contiguous drug exposures with a configurable persistence gap.

Index date The cohort entry date for a person; most criteria are expressed as windows relative to the index date.

WebAPI The REST backend that ATLAS v3.0 talks to. WebAPI executes cohort generation, vocabulary search, and reporting against the underlying CDM databases.

Source / data source A configured CDM database. ATLAS v3.0 lists every source that WebAPI is connected to; reports and cohort generation always run against a chosen source.

Characterization An analysis that summarises clinical features of a target cohort, optionally compared to outcomes, using a set of feature analyses.

Feature analysis A reusable definition of a covariate or summary statistic (e.g. "drug exposure in the 365 days before index"). Used by characterizations.

Cohort pathway A visualisation of the order in which event cohorts occur for the persons in a target cohort.

Incidence rate The number of new outcome events per person-time-at-risk inside a target cohort.

Inclusion rule A criterion applied after the entry event that a person must satisfy to remain in the cohort.

Censoring event A clinical event that ends a person's cohort membership before the configured exit window would otherwise close it.

PHOEBE A vocabulary-recommendation engine packaged with WebAPI that suggests additional concepts likely to belong in a concept set, drawn from co-occurrence patterns across an OMOP network. ATLAS v3.0 surfaces it as the **Recommend** tab inside the concept-set editor.

TrexSQL A query-level cache shared across users on a WebAPI host. The TrexSQL patient cache is what powers the live patient-count bar in the cohort builder; without a built cache, the bar is hidden or shows a "not ready" warning.

Patient cache The per-source, pre-aggregated dataset that descriptive reports and live patient counts read from. Built on demand from the configuration panel's **Cache** tab; rebuilt when the underlying CDM or vocabulary refreshes.

Previous runs The historical generations of an analysis, listed in the **Previous runs** dialog. The dialog is scoped to a single data source and opened from the **Show history** action on each row of the data-source run table at the top of the Characterization, Incidence Rate, and Pathway editors.

Subgroup analysis An overlapping selection of the population inside a characterization, defined like a small inclusion rule. Each subgroup adds an extra column to the result. (Internally referred to as *strata*; the UI label is "Subgroup analyses".)

Sunburst The radial diagram used to visualise cohort pathways: the index event sits at the centre, each subsequent step is a ring expanding outwards, wedge area is proportional to person count.

Achilles The OHDSI R routine that computes the descriptive statistics ATLAS reads from for the **Data Sources** reports. A source is "Achilles-processed" when those routines have been run against its results schema.

Athena OHDSI's vocabulary repository (`https://athena.ohdsi.org`). The vocabulary tables Athena publishes are loaded into WebAPI; Atlas's vocabulary search runs against that data, so two ATLAS instances on the same release but with different Athena snapshots can return slightly different concept IDs.

HADES The OHDSI R-package suite (*Health Analytics Data to Evidence Suite*). HADES packages such as `FeatureExtraction`, `CohortDiagnostics` and `PheValuator` consume the JSON ATLAS exports for analyses ATLAS itself does not run.

Bibliography

[1] Peter C. Austin. Balance diagnostics for comparing the distribution of baseline covariates between treatment groups in propensity-score matched samples. *Statistics in Medicine*, 28(25):3083–3107, 2009.

[2] EHDEN. Phenotype definition, characterisation and evaluation. EHDEN Academy course, https://www.ehden.eu/course_phenotypedefinitioncharacterisationevaluation/, 2024.

[3] Health Data Research UK. HDR UK phenotype library. https://phenotypes.healthdatagateway.org/, 2025.

[4] George Hripcsak, Jon D. Duke, Nigam H. Shah, Christian G. Reich, Vojtech Huser, Martijn J. Schuemie, Marc A. Suchard, Rae Woong Park, Ian Chi Kei Wong, Peter R. Rijnbeek, Johan van der Lei, Nicole Pratt, G. Niklas Norén, Yu-Chuan Li, Paul E. Stang, David Madigan, and Patrick B. Ryan. Observational health data sciences and informatics (OHDSI): Opportunities for observational researchers. *Studies in Health Technology and Informatics*, 216:574–578, 2015.

[5] George Hripcsak, Patrick B. Ryan, Jon D. Duke, Nigam H. Shah, Rae Woong Park, Vojtech Huser, Marc A. Suchard, Martijn J. Schuemie, Frank J. DeFalco, Adler Perotte, Juan M. Banda, Christian G. Reich, Lisa M. Schilling, Michael E. Matheny, Daniella Meeker, Nicole Pratt, and David Madigan. Characterizing treatment pathways at scale using the OHDSI network. *Proceedings of the National Academy of Sciences*, 113(27):7329–7336, 2016.

[6] George Hripcsak, Matthew E. Levine, Ning Shang, and Patrick B. Ryan. Effect of vocabulary mapping for conditions on phenotype cohorts. *Journal of the American Medical Informatics Association*, 25(12):1618–1625, 2018.

[7] Michael G. Kahn, Tiffany J. Callahan, Juliana Barnard, Alan E. Bauck, Jeff Brown, Bruce N. Davidson, Hossein Estiri, Carsten Goerg, Erin Holve, Steven G. Johnson, Siaw-Teng Liaw, Marianne Hamilton-Lopez, Daniella Meeker, Toan C. Ong, Patrick Ryan, Ning Shang, Nicole G. Weiskopf, Chunhua Weng, Meredith N. Zozus, and Lisa Schilling. A harmonized data quality assessment terminology and framework for the secondary use of electronic health record data. *eGEMs (Generating Evidence & Methods to improve patient outcomes)*, 4(1):18, 2016.

[8] John Leider and contributors. Vuetify: Material design component framework. https://vuetifyjs.com/.

[9] OHDSI. OHDSI achilles. https://github.com/OHDSI/Achilles, .

[10] OHDSI. Athena OHDSI vocabulary repository. https://athena.ohdsi.org/, .

[11] OHDSI. Atlas v3.0 source repository. https://github.com/OHDSI/Atlas3, .

[12] OHDSI. Ohdsi atlas (production, atlas 2.x). https://github.com/OHDSI/Atlas, .

[13] OHDSI. OHDSI broadsea. https://github.com/OHDSI/Broadsea, .

[14] OHDSI. OHDSI data quality dashboard (DQD). https://github.com/OHDSI/DataQualityDashboard, .

[15] OHDSI. OHDSI FeatureExtraction r package. https://ohdsi.github.io/FeatureExtraction/, .

[16] OHDSI. OMOP common data model specification. https://ohdsi.github.io/CommonDataModel/, .

[17] OHDSI. OHDSI phenotype library. https://github.com/OHDSI/PhenotypeLibrary, .

[18] OHDSI. PheValuator r package. https://ohdsi.github.io/PheValuator/, .

[19] OHDSI. Ohdsi webapi. https://github.com/OHDSI/WebAPI, .

[20] OHDSI. *The Book of OHDSI: Observational Health Data Sciences and Informatics*. OHDSI Collaborators, 2021. URL https://ohdsi.github.io/TheBookOfOhdsi/. Open-access community handbook.

[21] OHDSI. Ohdsi atlas wiki. https://github.com/OHDSI/Atlas/wiki, 2024.

[22] OHDSI. The book of OHDSI, chapter 16: Clinical validity. https://ohdsi.github.io/TheBookOfOhdsi/ClinicalValidity.html, 2024.

[23] OHDSI. Ohdsi hades (health analytics data-to-evidence suite). https://ohdsi.github.io/Hades/, 2024. R packages for cohort generation, characterization, population-level estimation, and patient-level prediction.

[24] OHDSI. Phenotype Phebruary. Annual community phenotyping sprint, https://www.ohdsi.org/phenotype-phebruary-2024/, 2024.

[25] OHDSI. OHDSI themis: Conventions for the omop common data model. https://github.com/OHDSI/Themis, 2024.

[26] OHDSI. Cohort definition submission requirements — OHDSI phenotype library. https://ohdsi.github.io/PhenotypeLibrary/articles/CohortDefinitionSubmissionRequirements.html, 2025.

[27] OHDSI. Evaluating phenotype algorithms (PheValuator vignette). https://ohdsi.github.io/PheValuator/articles/EvaluatingPhenotypeAlgorithms.html, 2025.

[28] OHDSI HADES. CohortDiagnostics: An r package for performing diagnostics on cohort definitions. https://ohdsi.github.io/CohortDiagnostics/, 2025.

[29] J. Marc Overhage, Patrick B. Ryan, Christian G. Reich, Abraham G. Hartzema, and Paul E. Stang. Validation of a common data model for active safety surveillance

research. *Journal of the American Medical Informatics Association*, 19(1):54–60, 2012.

[30] Gowtham A. Rao, Martijn J. Schuemie, Azza Shoaibi, et al. CohortDiagnostics: Phenotype evaluation across a network of observational data sources using population-level characterization. *PLOS ONE*, 2025. doi: 10.1371/journal.pone.0310634.

[31] William A. Ray. Evaluating medication effects outside of clinical trials: New-user designs. *American Journal of Epidemiology*, 158(9):915–920, 2003.

[32] Regenstrief Institute. LOINC: Logical observation identifiers names and codes. https://loinc.org/.

[33] Jenna M. Reps, Martijn J. Schuemie, Marc A. Suchard, Patrick B. Ryan, and Peter R. Rijnbeek. Design and implementation of a standardized framework to generate and evaluate patient-level prediction models using observational healthcare data. *Journal of the American Medical Informatics Association*, 25(8):969–975, 2018.

[34] Martijn J. Schuemie, George Hripcsak, Patrick B. Ryan, David Madigan, and Marc A. Suchard. Empirical confidence interval calibration for population-level effect estimation studies in observational healthcare data. *Proceedings of the National Academy of Sciences*, 115(11):2571–2577, 2018.

[35] SNOMED International. SNOMED CT. https://www.snomed.org/.

[36] Samy Suissa. Immortal time bias in pharmaco-epidemiology. *American Journal of Epidemiology*, 167(4):492–499, 2008.

[37] Jill N. Swerdel, George Hripcsak, and Patrick B. Ryan. PheValuator: Development and evaluation of a phenotype algorithm evaluator. *Journal of Biomedical Informatics*, 97:103258, 2019. doi: 10.1016/j.jbi.2019.103258.

[38] Joel N. Swerdel, Martijn Schuemie, Gayle Murray, and Patrick B. Ryan. PheValuator 2.0: Methodological improvements for the PheValuator approach to semi-automated phenotype algorithm evaluation. *Journal of Biomedical Informatics*, 135:104177, 2022. doi: 10.1016/j.jbi.2022.104177.

[39] Joel N. Swerdel, Martijn Schuemie, Gayle Murray, and Patrick B. Ryan. Comparing broad and narrow phenotype algorithms: differences in performance characteristics and immortal time incurred. *JAMIA Open*, 2024. doi: 10.1093/jamiaopen/ooae002.

[40] U.S. National Library of Medicine. RxNorm. https://www.nlm.nih.gov/research/umls/rxnorm/.

[41] Erica A. Voss, Rupa Makadia, Amy Matcho, Qianli Ma, Christopher Knoll, Martijn Schuemie, Frank J. DeFalco, Ajit Londhe, Vivienne Zhu, and Patrick B. Ryan. Feasibility and utility of applications of the common data model to multiple, dis-

parate observational health databases. *Journal of the American Medical Informatics Association*, 22(3):553–564, 2015.

[42] Kazuki Yoshida, Daniel H. Solomon, and Seoyoung C. Kim. The active comparator, new user study design in pharmacoepidemiology: established but no longer the standard? *Current Epidemiology Reports*, 2:221–228, 2015.

[43] Evan You and contributors. Vue.js: The progressive javascript framework. https://vuejs.org/.